

PROGRAMOWANIE OBIEKTOWE**Projekt**

Wydział Informatyki i Telekomunikacji	Kierunek: Informatyka Techniczna
Grupa zajęciowa: Poniedziałek godz. 9.15	Semestr: 2025/26 LATO
Nazwisko i Imię: Kanecki Jakub	Nr indeksu: 293485
Nazwisko i Imię: Łazarski Dawid	Nr indeksu: 290307
Nazwisko i Imię:	Nr indeksu:
Nr. grupy projektowej: 16	
Prowadzący: mgr inż. Karol Puchała	

TEMAT:

„System zarządzania parkingiem”

OCENA:**PUNKTY:**

Data:

Standardowa zawartość sprawozdania

1) Spis narzędzi użytych przy tworzeniu projektu.

Do utworzenia projektu wykorzystano język programowania Java, bibliotekę graficzną Swing oraz użyto NetBeans IDE.

2) Założenia i opis funkcjonalny programu.

Program ma za zadanie symulować obsługę parkingu miejskiego. Każdy pojazd, który wjedzie na parking, zostanie zidentyfikowany, poprzez numer rejestracyjny(w przypadku samochodów) lub bilet(w przypadku rowerów).

Opis funkcjonalny:

System umożliwia podstawowe operacje związane z obsługą parkingu:

- rejestracja wjazdu pojazdu (samochód, rower) na parking,
- stała rejestracja samochodów na podstawie ich numerów rejestracyjnych lub wygenerowanych kodów(w przypadku rowerów)
- rejestracja wyjazdu pojazdu,
- automatyczne przypisywanie miejsca parkingowego uwzględniające przydzielanie miejsc z ładowarkami dla pojazdów elektrycznych,
- sprawdzanie dostępnych miejsc w czasie rzeczywistym,
- blokowanie/odblokowywanie miejsc parkingowych (np. awaria lub rezerwacja),
- możliwość zablokowania wjazdu dla podanych numerów rejestracyjnych,
- generowanie raportów (ilość pojazdów, czas zajęcia miejsc, statystyki).

3) Klasy(chodzi o klasy dodane przez siebie, a nie przez automat kreatora)

Pakiet `com.systemzarządzaniaparkingiem.model` :

- Pojazd – abstrakcyjna klasa bazowa pojazdu.
- Samochod – klasa samochodu, dziedziczy po Pojazd identyfikowany numerem rejestracyjnym
- Rower – klasa roweru, dziedziczy po Pojazd, identyfikowany kwitem;
- Wlasciciel – dane właściciela tablicy: imię i nazwisko, e-mail, telefon.
- STATUS_MIEJSCA – typ wyliczeniowy (enum) ze stanami miejsca: DOSTĘPNE, ZAJĘTE, ZABLOKOWANE, ZAREZERWOWANE.
- MiejsceParkingowe – abstrakcyjna klasa bazowa miejsca; przechowuje numer, status i aktywną sesję.
- MiejsceSamochodowe – miejsce dla samochodu, opcjonalnie wyposażone w ładowarkę EV.
- MiejsceRowerowe – miejsce dla roweru, opcjonalnie wyposażone w blokadę.

- SesjaParkingowa – pojedyncza sesja postoju: czas wjazdu, planowany/rzeczywisty wyjazd, numer kwitu.
- ZarejestrowaneTablice – rejestr tablic i właścicieli oraz „czarna lista” zablokowanych tablic.
- Logi – dziennik zdarzeń systemu parkingowego
- IRaporty – interfejs definiujący sposób raportowania
- Raporty – implementacja interfejsu IRaporty; zlicza miejsca i pojazdy oraz generuje raport tekstowy.
- Parking – model parkingu, centralna klasa spinająca całą logikę parkingu

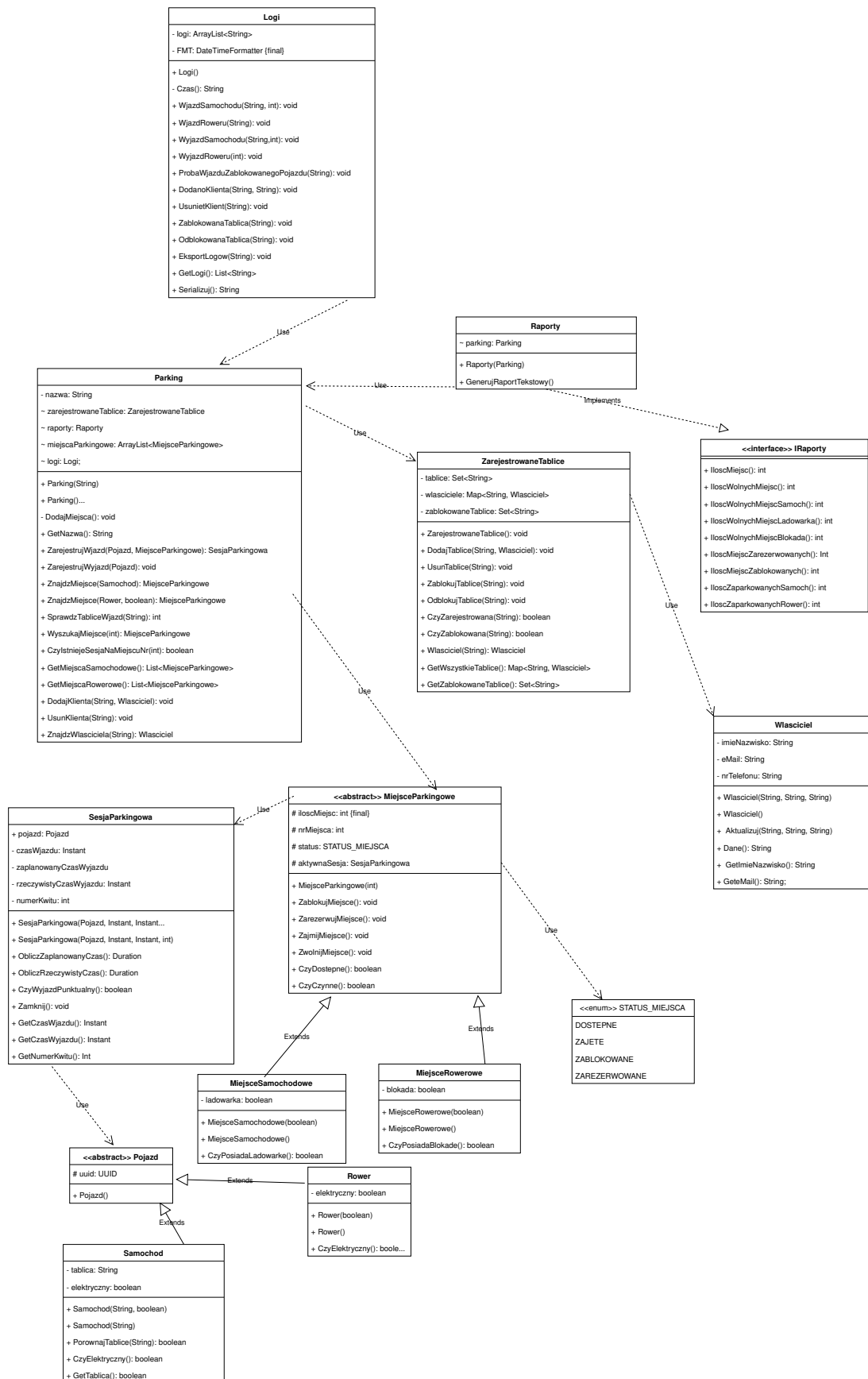
Pakiet `com.systemzarzadzaniaparkingiem.widok` :

- MainWindow – okno główne (JFrame): wizualizacja miejsc, menu, panel operacji i panel raportu.
- DialogWjazdSamochodu – okno rejestracji wjazdu samochodu (tablica, EV, czas postoju).
- DialogWjazdRoweru – okno rejestracji wjazdu roweru z generowaniem kwitu.
- DialogWyjazd – okno rejestracji wyjazdu – po tablicy (samochód) lub numerze kwitu (rower).
- DialogDodajKlienta – okno rejestracji tablicy wraz z danymi właściciela.
- DialogWyszukajKlienta – okno wyszukiwania, edycji i usuwania zarejestrowanych klientów.
- DialogCzarnaLista – okno zarządzania czarną listą tablic.
- DialogEksportLogow – okno podglądu i eksportu logów systemu do pliku TXT.

Pakiet główny `com.systemzarzadzaniaparkingiem` :

- SystemZarzadzaniaParkingiem – klasa startowa z metodą `main()`

a) Diagram UML własnych klas (na następnej stronie):



b) Kod własnych klas

// tylko zawartość **plików nagłówkowych** *.h/ *.hpp (C++) – zarówno deklarowane zmienne jak i prototypy funkcji powinny być skomentowane

//pakiet z własnymi klasami (Java)

Dla Pakietu **com.systemzarzadzaniaparkingiem.model**:

STATUS_MIEJSCA.java

```
/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public enum STATUS_MIEJSCA { // zrobione
    DOSTEPNE, ZAJETE, ZABLOKOWANE, ZAREZERWOWANE
}
```

Pojazd.java

```
/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public abstract class Pojazd {
    protected UUID uuid;
    public Pojazd() { ... }
}
```

Samochod.java

```
/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Samochod extends Pojazd {
    private String tablica;
    private boolean elektryczny;

    public Samochod(String tablica, boolean elektryczny) { ... }

    public Samochod(String tablica) { ... }

    public boolean PorownajTablice(String tablica2) { ... }
    public boolean CzyElektryczny() { ... }
    public String GetTablica() { ... }
}
```

Rower.java

```
/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Rower extends Pojazd {
    private boolean elektryczny;

    public Rower(boolean elektryczny) { ... }
```

```

    public Rower() { ... }
    public boolean CzyElektryczny() { ... }
}

```

Wlasciciel.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Wlasciciel {
    private String imieNazwisko;
    private String eMail;
    private String nrTelefonu;

    public Wlasciciel(String imieNazwisko, String eMail, String nrTelefonu) { ... }

    public Wlasciciel() { ... }

    // Odczyt danych - konieczne do wyświetlania w GUI i wyszukiwania
    public String GetImieNazwisko() { ... }
    public String GeteMail() { ... }
    public String GetNrTelefonu() { ... }

    public void Aktualizuj(String imieNazwisko, String eMail, String nrTelefonu) { ... }

    public String Dane() { ... }
}

```

SesjaParkingowa.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class SesjaParkingowa {
    public Pojazd pojazd;
    private Instant czasWjazdu;
    private Instant zaplanowanyCzasWyjazdu;
    private Instant rzeczywistyCzasWyjazdu;
    private int numerKwitu; // dla rowerów

    public SesjaParkingowa(Pojazd pojazd, Instant czasWjazdu, Instant
zaplanowanyCzasWyjazdu) { ... }

    public SesjaParkingowa(Pojazd pojazd, Instant czasWjazdu, Instant
zaplanowanyCzasWyjazdu, int numerKwitu) { ... }

    public Duration ObliczZaplanowanyCzas() { ... }

    public Duration ObliczRzeczywistyCzas() { ... }

    public void Zamknij() { ... }

    public boolean CzyWyjazdPunktualny() { ... }

    public Instant getCzasWjazdu() { ... }
    public Instant getZaplanowanyCzasWyjazdu() { ... }
}

```

```
    public int getNumerKwitu() { ... }  
}
```

MiejsceParkingowe.java

```
/**  
 *  
 * @author Jakub Kanecki, Dawid Lazarski  
 */  
public abstract class MiejsceParkingowe {  
    protected static int iloscMiejsc = 0;  
    protected int nrMiejsc;  
    protected STATUS_MIEJSCA status;  
    protected SesjaParkingowa aktywnaSesja;  
  
    public MiejsceParkingowe() { ... }  
  
    public static void resetLicznik() { ... }  
  
    public void ZablokujMiejsce() { ... }  
    public void ZarezerwujMiejsce() { ... }  
    public void OdblokujMiejsce() { ... }  
  
    public void ZajmijMiejsce(SesjaParkingowa sesja) { ... }  
  
    public void ZwolnijMiejsce() { ... }  
  
    public boolean CzyDostepne() { ... }  
    public boolean CzyCzynne() { ... }  
    public boolean CzyZarezerwowane() { ... }  
    public boolean CzyZajete() { ... }  
    public boolean CzyPosiadaLadowarke() { ... }  
    public boolean CzyPosiadaBlokade() { ... }  
    public boolean PorownajId(int id) { ... }  
  
    public int getNrMiejsc() { ... }  
    public STATUS_MIEJSCA getStatus() { ... }  
    public SesjaParkingowa getAktywnaSesja() { ... }  
}
```

MiejsceSamochodowe.java

```
/**  
 *  
 * @author Jakub Kanecki, Dawid Lazarski  
 */  
public class MiejsceSamochodowe extends MiejsceParkingowe {  
    private boolean ladowarka;  
  
    public MiejsceSamochodowe(boolean ladowarka) { ... }  
    public MiejsceSamochodowe() { ... }  
  
    @Override public boolean CzyPosiadaLadowarke() { return this.ladowarka; }  
}
```

MiejsceRowerowe.java


```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class MiejsceRowerowe extends MiejsceParkingowe {
    private boolean blokada;

    public MiejsceRowerowe(boolean blokada) { ... }
    public MiejsceRowerowe() { ... }

    @Override public boolean CzyPosiadaBlokade() { return this.blokada; }
}

```

ZarejestrowaneTablice.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class ZarejestrowaneTablice {
    private HashSet<String> tablice;
    private HashMap<String, Wlasciciel> wlasciciele;
    private HashSet<String> zablokowaneTablice;

    public ZarejestrowaneTablice() { ... }

    public void DodajTablice(String tablica, Wlasciciel wlasciciel) { ... }

    public void UsunTablice(String tablica) { ... }

    public void ZablockujTablice(String tablica) { ... }

    public void OdblockujTablice(String tablica) { ... }

    public boolean CzyZarejestrowana(String tablica) { ... }

    public boolean CzyZablokowana(String tablica) { ... }

    public Wlasciciel Wlasciciel(String tablica) { ... }

    public Map<String, Wlasciciel> GetWszystkieTablice() { ... }
    public Set<String> GetZablokowaneTablice() { ... }
}

```

Logi.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Logi {
    private ArrayList<String> logi;
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss").withZone(ZoneId.systemDefault());

    public Logi() { ... }

    private String Czas() { ... }
}

```

```

    public void WjazdSamochodu(String tab, int miejsce) { ... }

    public void WjazdRoweru(int miejsce) { ... }

    public void WyjazdSamochodu(String tab, int miejsce) { ... }

    public void WyjazdRoweru(int miejsce) { ... }

    public void ProbaWjazduZablokowanegoPojazdu(String tab) { ... }

    public void DodanoKlienta(String tab, String dane) { ... }

    public void UsunietKlient(String tab) { ... }

    public void ZablokowanaTablica(String tab) { ... }

    public void OdblokowanaTablica(String tab) { ... }

    public void EksportLogow(String sciezka) { ... }

    public List<String> GetLogi() { ... }

    public String Serializuj() { ... }
}

```

IRaporty.java

```

/**
 * @author Jakub Kanecki, Dawid Lazarski
 */
public interface IRaporty { // do zmian w UML
    // miejsca
    public int IloscMiejsc();
    public int IloscWolnychMiejsc();
    public int IloscWolnychMiejscSamoch();
    public int IloscWolnychMiejscLadowarka();
    public int IloscWolnychMiejscRower();
    public int IloscWolnychMiejscBlokada();
    public int IloscMiejscZarezerwowanych();
    public int IloscMiejscZablokowanych();

    // pojazdy
    // IloscZaparkowanychPojazdow do wyliczenia przez IloscMiejsc() - IloscWolnychMiejsc()
    public int IloscZaparkowanychSamoch();
    public int IloscZaparkowanychRower();
    // raport tekstowy
}

```

Raporty.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Raporty implements IRaporty {
    Parking parking;
}

```

```

    Raporty(Parking parking) { ... }
    @Override
    public int IloscMiejsc() { ... }
    @Override
    public int IloscWolnychMiejsc() { ... }
    @Override
    public int IloscWolnychMiejscSamoch() { ... }
    @Override
    public int IloscWolnychMiejscLadowarka() { ... }
    @Override
    public int IloscWolnychMiejscRower() { ... }
    @Override
    public int IloscWolnychMiejscBlokada() { ... }
    @Override
    public int IloscMiejscZarezerwowanych() { ... }
    @Override
    public int IloscMiejscZablokowanych() { ... }
    @Override
    public int IloscZaparkowanychSamoch() { ... }
    @Override
    public int IloscZaparkowanychRower() { ... }

    public String GenerujRaportTekstowy() { ... }
}

```

Parking.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Parking {
    private String nazwa;
    public ZarejestrowaneTablice zarejestrowaneTablice;
    public Raporty raporty;
    public ArrayList<MiejsceParkingowe> miejscaParkingowe;
    public Logi logi;

    public Parking(String nazwa) { ... }

    public Parking() { ... }

    private void DodajMiejsca() { ... }

    public String GetNazwa() { ... }

    // — operacje parkingowe —————

    public void ZarejestrujWjazd(Pojazd pojazd, MiejsceParkingowe miejsce, Instant
zakonczeniePostoju) { ... }

    public void ZarejestrujWyjazd(MiejsceParkingowe miejsce) { ... }

    public MiejsceParkingowe ZnajdzMiejsce(Samochod pojazd) { ... }

    public MiejsceParkingowe ZnajdzMiejsce(Rower pojazd, boolean wlasnaBlokada) { ... }

    /** Zwraca -1 jeśli zablokowana, 1 jeśli zarejestrowana, 0 jeśli nieznana */
    public int sprawdzTabliceWjazd(String tablica) { ... }
}

```

```

public MiejsceParkingowe WyszukajMiejsce(int id) { ... }

public boolean CzyIstniejeSesjaNaMiejscuNr(int id) { ... }

public List<MiejsceParkingowe> GetMiejscaSamochodowe() { ... }

public List<MiejsceParkingowe> GetMiejscaRowerowe() { ... }

public MiejsceParkingowe ZnajdzMiejscePojazdaZTabl(String tablica) { ... }

public void DodajKlienta(String tablica, Wlasciciel wlasciciel) { ... }

public void UsunKlienta(String tablica) { ... }

public Wlasciciel ZnajdzWlasciciela(String tablica) { ... }

}

```

Dla pakietu **com.systemzarzadzaniaparkingiem.widok:**

MainWindow.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class MainWindow extends JFrame {

    private final Parking parking;
    private JPanel panelSamochodowe;
    private JPanel panelRowerowe;
    private JLabel labelRaport;
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("HH:mm").withZone(ZoneId.systemDefault());

    // Kolory
    private static final Color CLR_WOLNE      = new Color(180, 230, 180);
    private static final Color CLR_ZAJETE     = new Color(230, 100, 100);
    private static final Color CLR_ZABLOK     = new Color(200, 200, 200);
    private static final Color CLR_LADOW      = new Color(180, 210, 255);
    private static final Color CLR_ROWER_BL   = new Color(255, 220, 140);
    private static final Color CLR_TLO_SEKCJI = new Color(240, 240, 245);
    private static final Color CLR_NAGLOWEK   = new Color(50, 60, 80);

    public MainWindow() { ... }

    private void buildUI() { ... }

    private JLabel naglowekSekcji(String tekst) { ... }

    private JPanel buildPanelPrzyciskow() { ... }

    private JPanel buildLegenda() { ... }

    private JPanel legendaKlocek(Color kolor, String opis) { ... }
}

```

```

    public void odswiezWidok() { ... }

    private void odswiezSekcje() { ... }

    private JPanel buildKafelekSamochod(MiejsceParkingowe m) { ... }

    private JPanel buildKafelekRower(MiejsceParkingowe m) { ... }

    private JLabel boldLabel(String text, int size) { ... }

    private void odswiezRaport() { ... }

    /**
     * FlowLayout z zawijaniem – miejsca parkingowe układają się w wiele rzędów
     * przy zmianie szerokości okna.
     */
    static class WrapLayout extends FlowLayout {
        WrapLayout(int align, int hgap, int vgap) { ... }

        @Override
        public Dimension preferredLayoutSize(Container target) { ... }

        @Override
        public Dimension minimumLayoutSize(Container target) { ... }

        private Dimension layoutSize(Container target, boolean preferred) { ... }
    }
}

```

DialogWjazdSamochodu.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWjazdSamochodu extends JDialog {
    private final Parking parking;
    private JTextField fieldTablica;
    private JCheckBox checkElektryczny;
    private JSpinner spinnerGodziny;
    private JLabel labelWynik;

    public DialogWjazdSamochodu(Frame owner, Parking parking) { ... }

    private void buildUI() { ... }

    private void zarejestrujWjazd() { ... }
}

```

DialogWjazdRoweru.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWjazdRoweru extends JDialog {
    private final Parking parking;
    private JCheckBox checkWlasnaBlokada;
}

```

```

        private JLabel labelWynik;

        public DialogWjazdRoweru(Frame owner, Parking parking) { ... }

        private void buildUI() { ... }

        private void zarejestrujWjazd() { ... }
    }

```

DialogWyjazd.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWyjazd extends JDialog {
    private final Parking parking;
    private JTextField fieldTablicaLubKwit;
    private JRadioButton rbSamochod, rbRower;
    private JLabel labelWynik;

    public DialogWyjazd(Frame owner, Parking parking) { ... }

    private void buildUI() { ... }

    private void zarejestrujWyjazd() { ... }
}

```

DialogDodajKlienta.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogDodajKlienta extends JDialog {
    private final Parking parking;
    private JTextField fieldTablica, fieldImie, fieldEmail, fieldTelefon;
    private JLabel labelWynik;

    public DialogDodajKlienta(Frame owner, Parking parking) { ... }

    private void buildUI() { ... }

    private void dodaj() { ... }
}

```

DialogWyszukajKlienta.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWyszukajKlienta extends JDialog {
    private final Parking parking;
    private JTextField fieldSzukaj;
    private DefaultTableModel tableModel;
    private JTable tabela;
}

```

```

    public DialogWyszukajKlienta(Frame owner, Parking parking) { ... }

    private void buildUI() { ... }

    private void odswiezTabele(String fraza) { ... }

    private void edytujWybrany() { ... }

    private void usunWybrany() { ... }
}

```

DialogCzarnaLista.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogCzarnaLista extends JDialog {
    private final Parking parking;
    private DefaultTableModel tableModel;
    private JTable tabela;
    private JTextField fieldSzukaj;

    public DialogCzarnaLista(Frame owner, Parking parking) { ... }

    private void buildUI() { ... }

    private void odswiezTabele(String fraza) { ... }

    private void usunWybrany() { ... }
}

```

DialogEksportLogow.java

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogEksportLogow extends JDialog {
    private final Parking parking;
    private JTextArea areaLogi;

    public DialogEksportLogow(Frame owner, Parking parking) { ... }

    private void buildUI() { ... }

    private void eksportuj() { ... }
}

```

Pakiet główny – klasa startowa:

SystemZarzadzaniaParkingiem.java

```

/**
 * @autorzy: Jakub Kanecki, Dawid Łazarski
 */

```

```
public class SystemZarzadzaniaParkingiem {

    public static void main(String[] args) { ... }

}
```

4) Schematy blokowe własnych funkcji C++ / Java

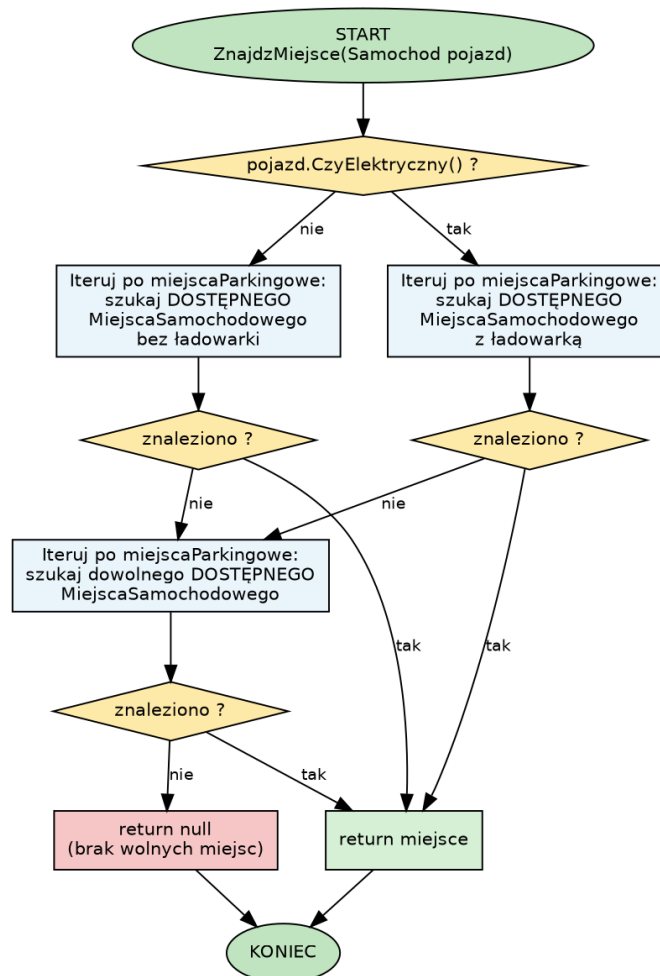
// Opisując własne funkcje podać: co funkcja robi, dane wejściowe, dane wyjściowe i wyniki, ewentualnie realizowane wzory lub algorytmy (np. qsort, minimalizacja czasu przejścia, minimalizacja liczby przeskoków itp.)

a) Parking.ZnajdźMiejsce(Samochod pojazd)

Co robi: wyszukuje optymalne wolne miejsce samochodowe. Dla pojazdu elektrycznego w pierwszej kolejności szuka miejsca z ładowarką, dla spalinowego – bez ładowarki; jeśli takiego brak, stosuje wspólny krok awaryjny i przydziela dowolne wolne miejsce samochodowe.

Dane wejściowe: obiekt Samochod (z informacją, czy jest elektryczny).

Dane wyjściowe / wynik: referencja do wolnego MiejsceParkingowe albo null, gdy brak wolnych miejsc.



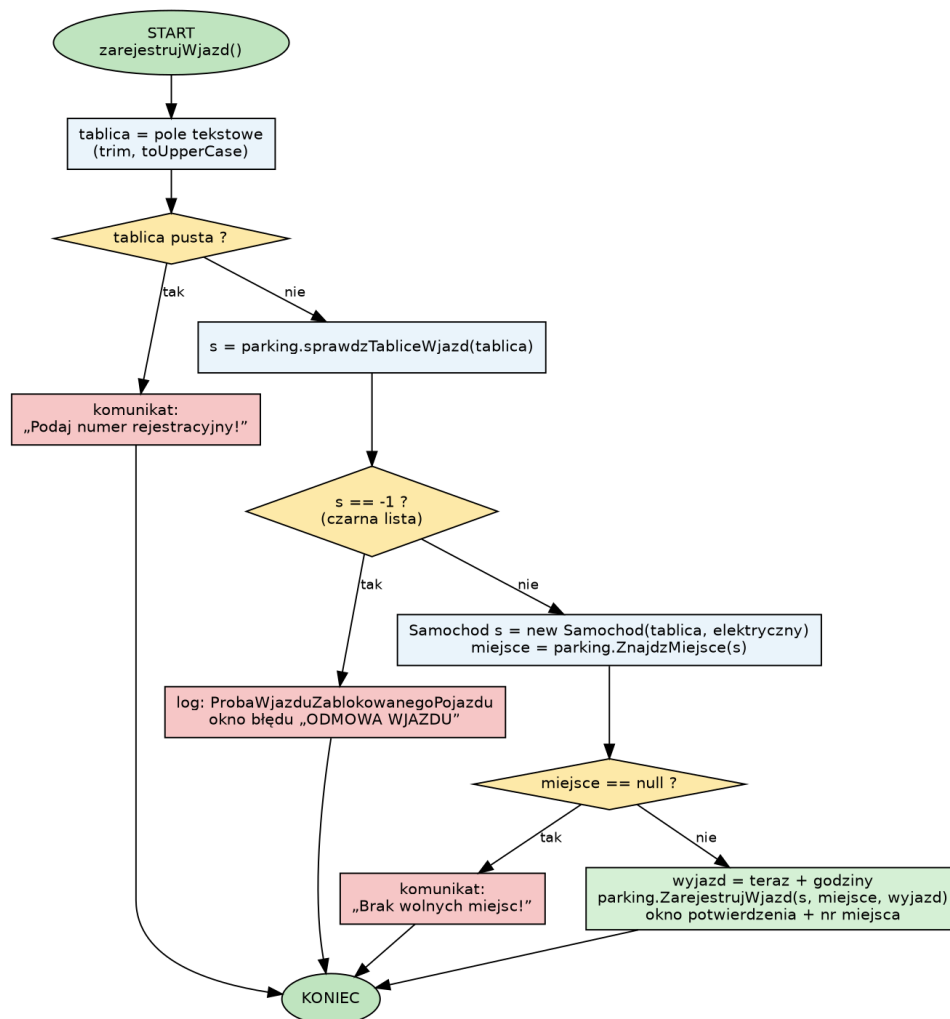
Rys. 1. Schemat blokowy funkcji `ZnajdzMiejsce(Samochod)` – priorytet miejsca z ładowarką.

b) `DialogWjazdSamochodu.zarejestrujWjazd()`

Co robi: obsługuje pełny scenariusz wjazdu samochodu: walidację pustego pola, sprawdzenie czarnej listy (metoda `sprawdzTabliceWjazd` zwraca -1/0/1), wyszukanie miejsca oraz zapis sesji wraz z logowaniem zdarzenia.

Dane wejściowe: numer rejestracyjny z pola tekstowego, flaga „elektryczny”, planowany czas postoju (godziny).

Dane wyjściowe / wynik: rejestracja sesji na przydzielonym miejscu i potwierdzenie, albo komunikat o błędzie/odmowie.



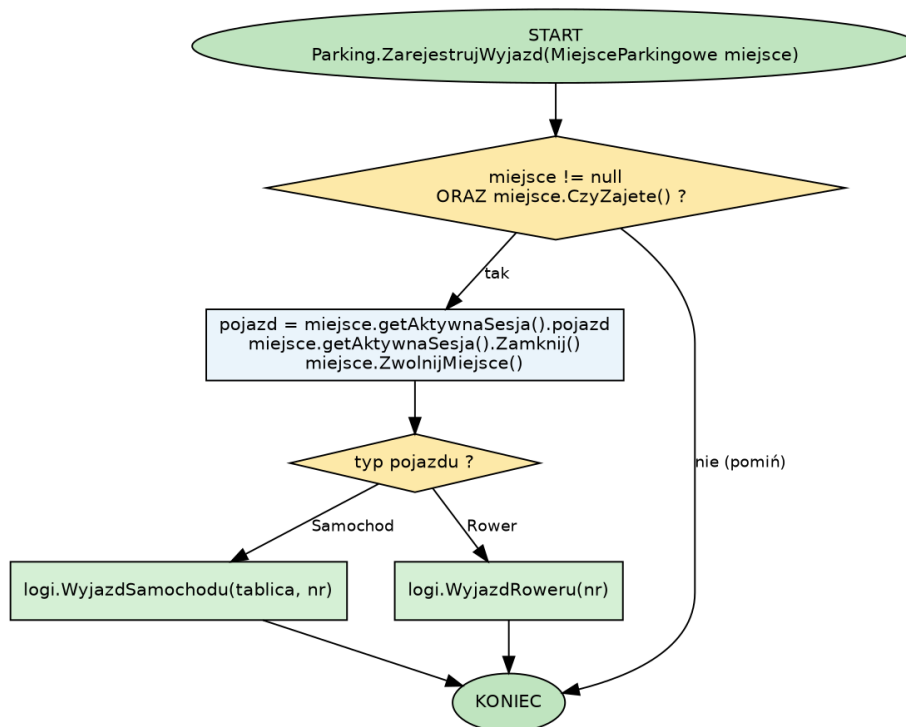
Rys. 2. Schemat blokowy obsługi wjazdu samochodu (walidacja → czarna lista → przydział miejsca).

c) `Parking.ZarejestrujWyjazd(MiejsceParkingowe miejsce)`

Co robi: kończy sesję postoju: weryfikuje, czy miejsce jest zajęte, zamyka sesję (zapis rzeczywistego czasu wyjazdu), zwalnia miejsce i – w zależności od typu pojazdu – dopisuje odpowiedni wpis do dziennika.

Dane wejściowe: miejsce parkingowe, które ma zostać zwolnione.

Dane wyjściowe / wynik: zamknięcie aktywnej sesji, zwolnienie miejsca i wpis do logów (rozróżnienie samochód/rower).



Rys. 3. Schemat blokowy funkcji ZarejestrujWyjazd – zamknięcie sesji i aktualizacja logów.

5) Opis użytkowy programu C++ /Java

//opis jak poruszać się po programie wraz z zrzutami ekranu (zwłaszcza opis menu), wymagania techniczne i uwagi instalacyjne (w przypadku skomplikowanej instalacji, projekt można wzbogacić opracowanym instalatorem).

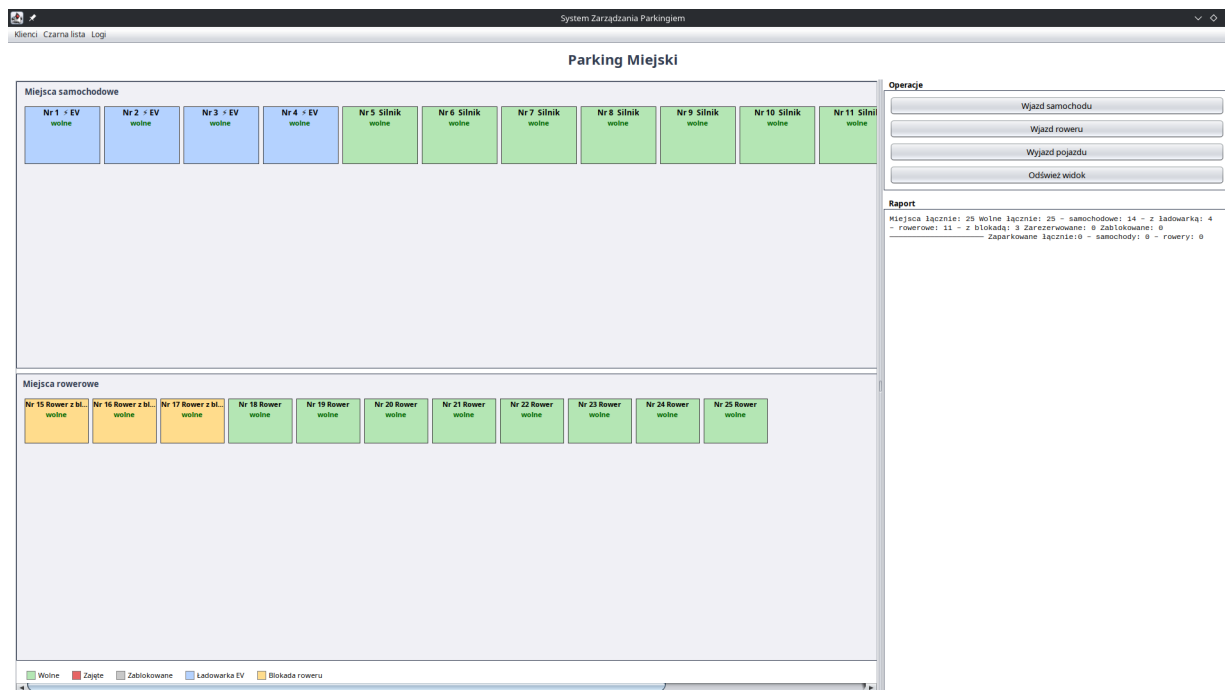
Wymagania i uruchomienie

- Do uruchomienia potrzebna jest Java (JDK 17 lub nowsza).
- Interfejs graficzny korzysta z biblioteki Swing, która jest częścią Javy – nie trzeba instalować nic dodatkowego.
- Program uruchamia się przez klasę startową SystemZarzadzaniaParkiniem (metoda main).
- Projekt był pisany w środowisku NetBeans; można go otworzyć i uruchomić przyciskiem Run.
- Można uruchomić również (w przypadku posiadania mavena) ten projekt będąc w katalogu kodu źródłowego i uruchamiając za pomocą:

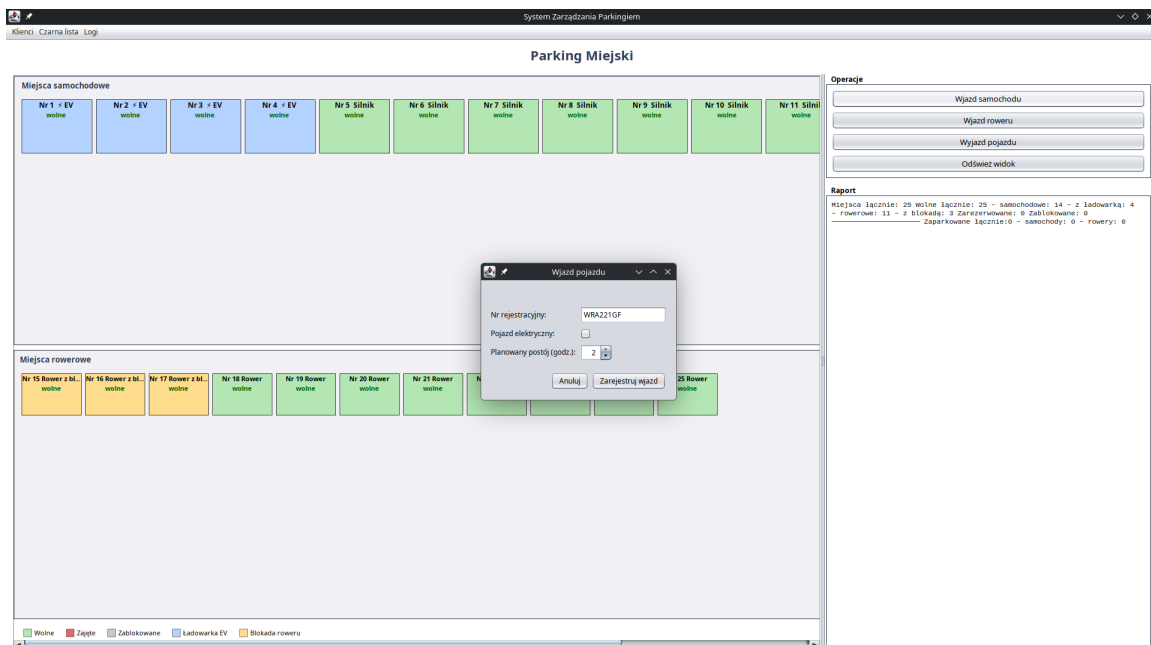
```
mvn clean package
mvn exec:java
```

Sposób obsługi programu

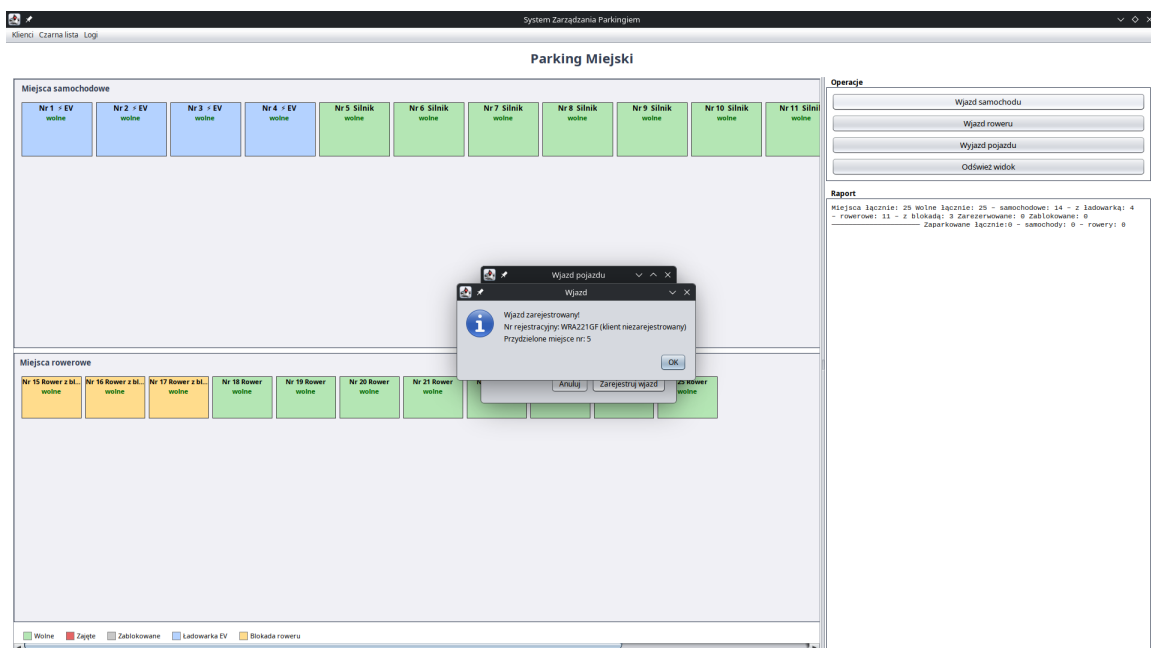
Po uruchomieniu pojawia się okno główne podzielone na sekcję miejsc samochodowych i rowerowych, panel operacji (po prawej) oraz panel raportu z bieżącymi statystykami. Górne menu udostępnia zarządzanie klientami, czarną listą i logami. Kolor kafelka informuje o stanie miejsca – legenda znajduje się na dole okna.



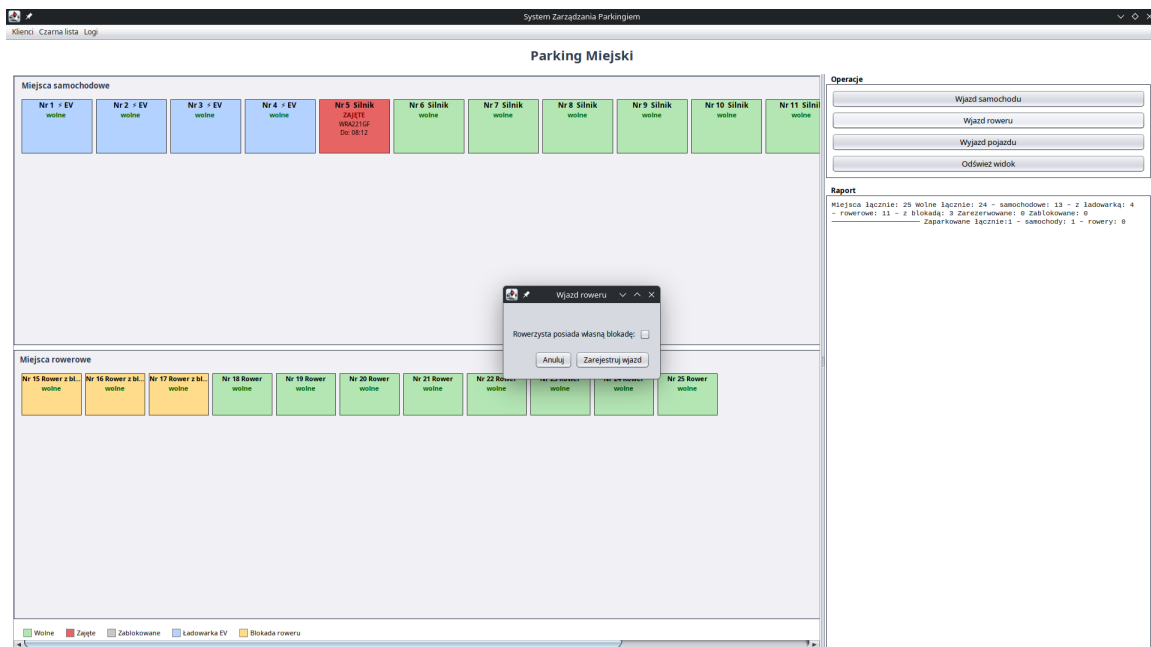
Rys. 4. Okno główne w stanie początkowym – wszystkie miejsca wolne. Widoczne sekcje miejsc samochodowych i rowerowych, panel „Operacje”, panel „Raport” oraz legenda kolorów.



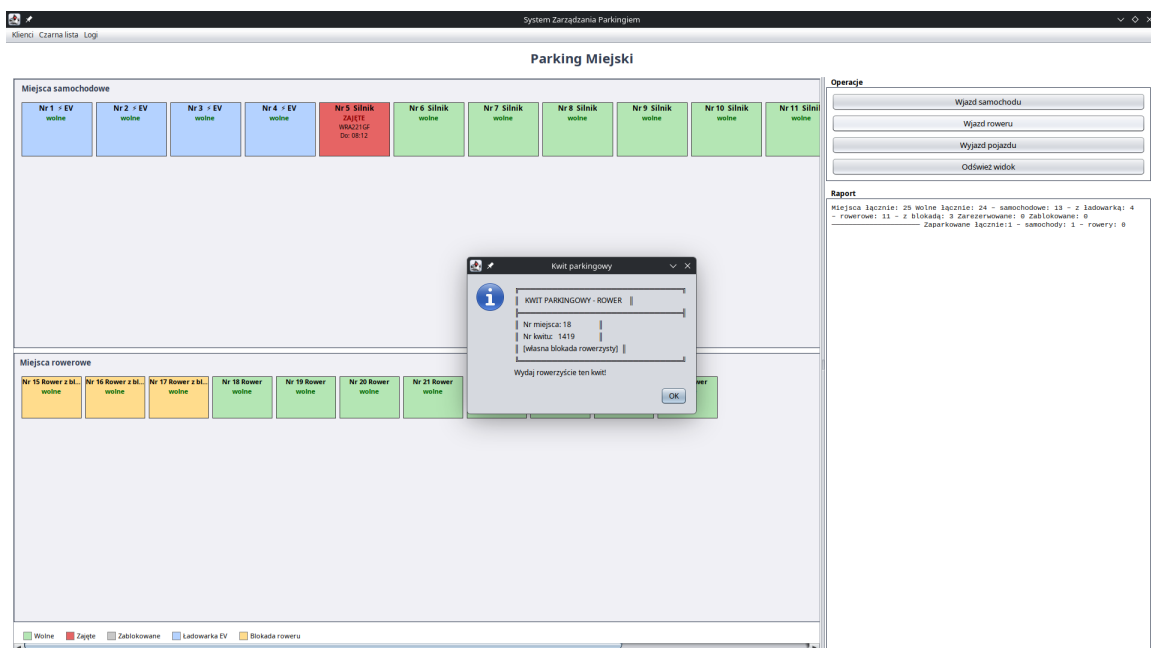
Rys. 5. Okno „Wjazd samochodu” – formularz z numerem rejestracyjnym, opcją pojazdu elektrycznego i planowanym czasem postoju (spinner godzin).



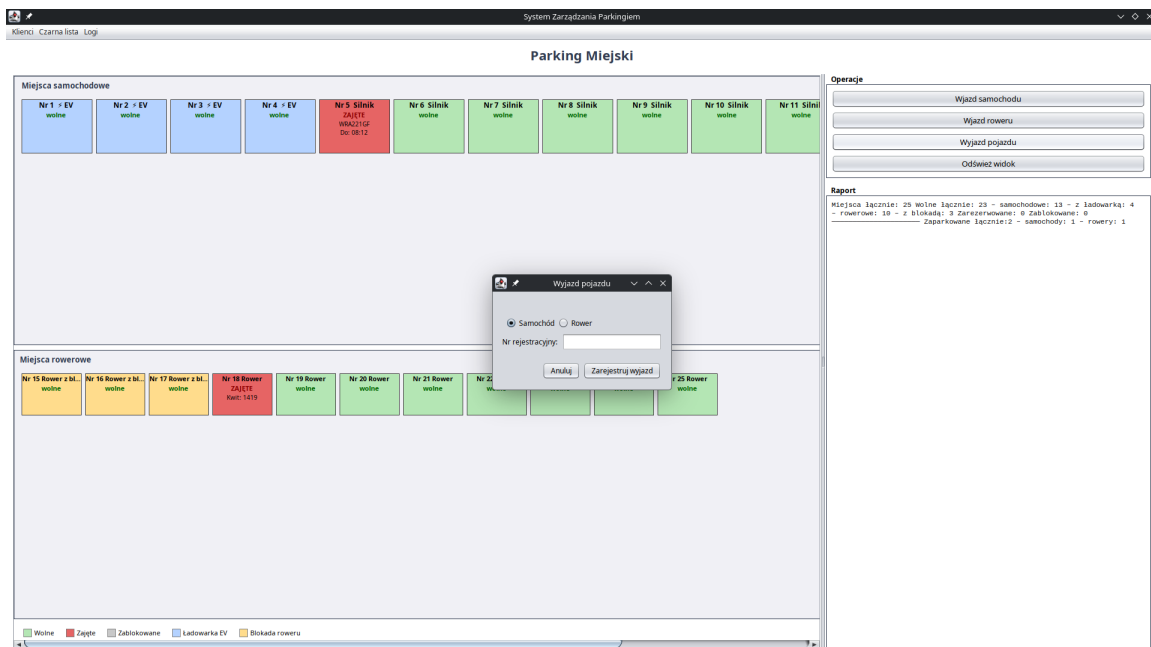
Rys. 6. Potwierdzenie rejestracji wjazdu i informacja o przydzielonym miejscu. Kafelki zajętego miejsca zmienia kolor na czerwony i pokazuje tablicę oraz planowaną godzinę wyjazdu.



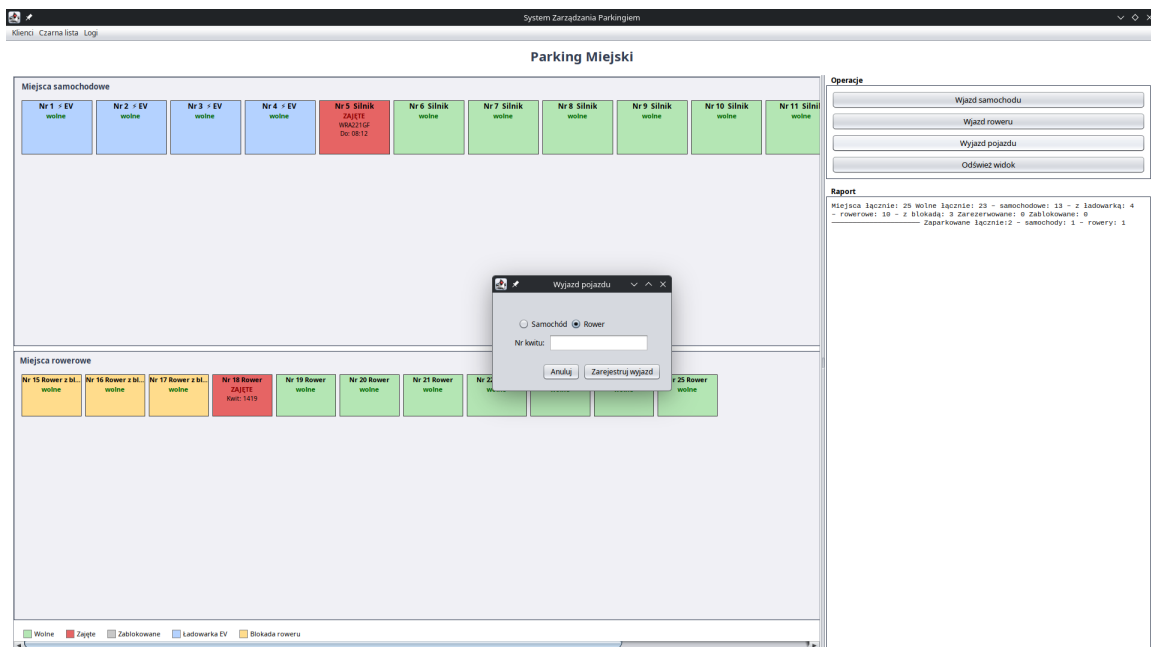
Rys. 7. Okno „Wjazd roweru” z opcją oznaczającą, że rowerzysta posiada własną blokadę (wpływa to na dobór miejsca).



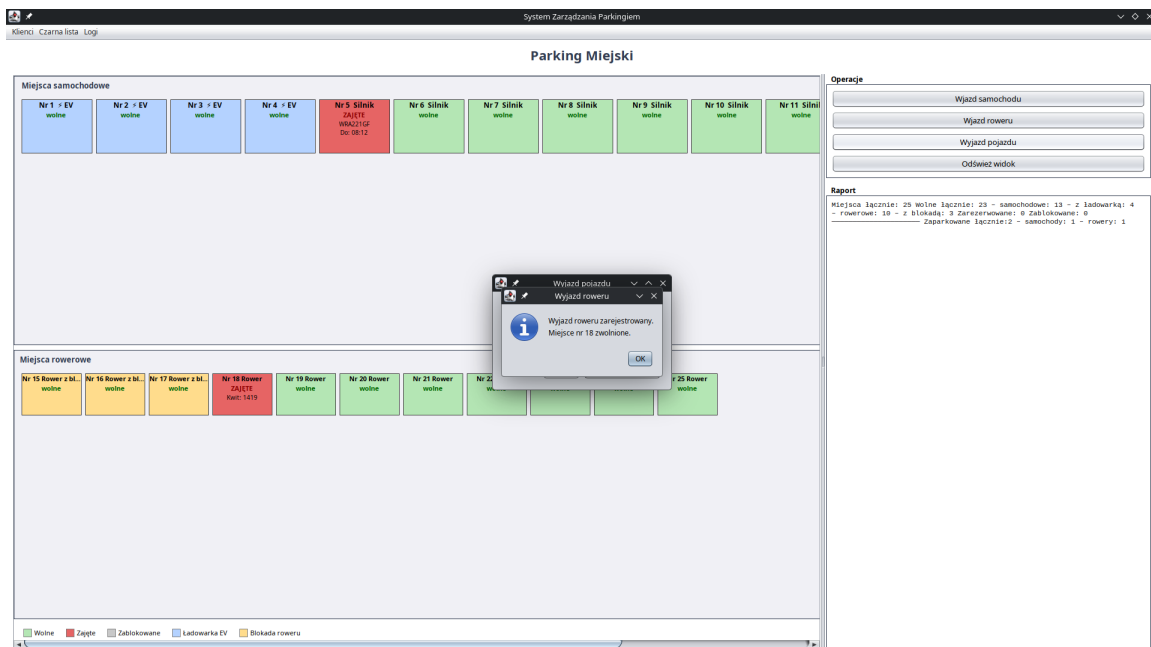
Rys. 8. Wygenerowany kwit parkingowy dla roweru z numerem miejsca i numerem kwitu, który jest potrzebny przy wyjeździe.



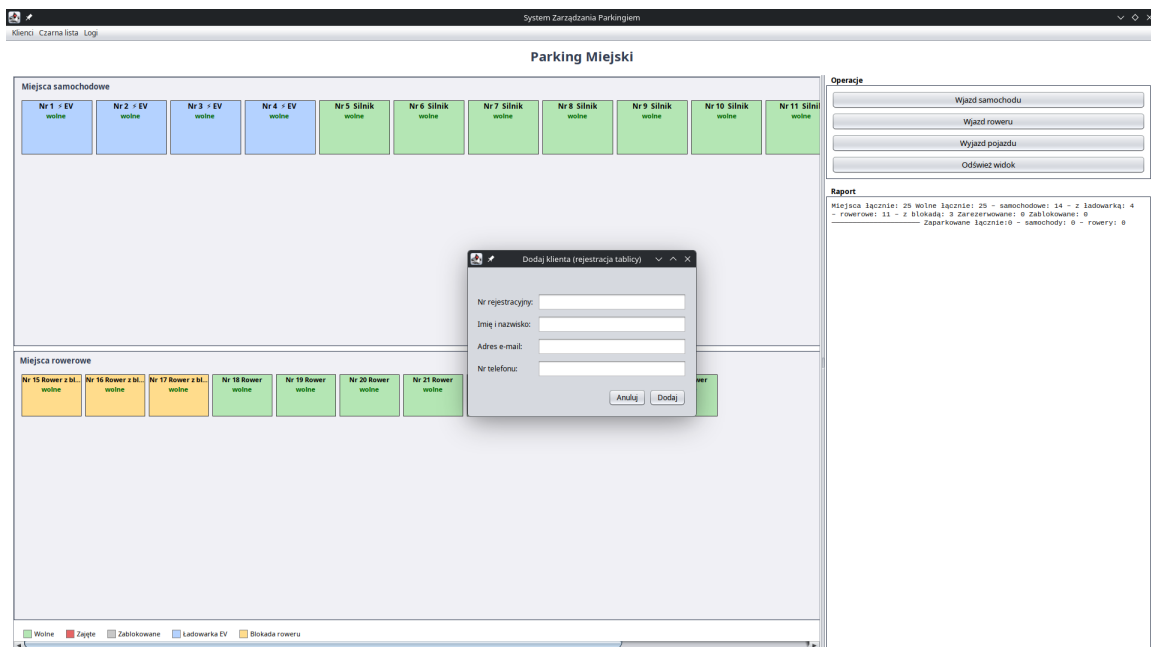
Rys. 9. Okno „Wjazd pojazdu” w trybie samochodu – pojazd wyszukiwany jest po numerze rejestracyjnym.



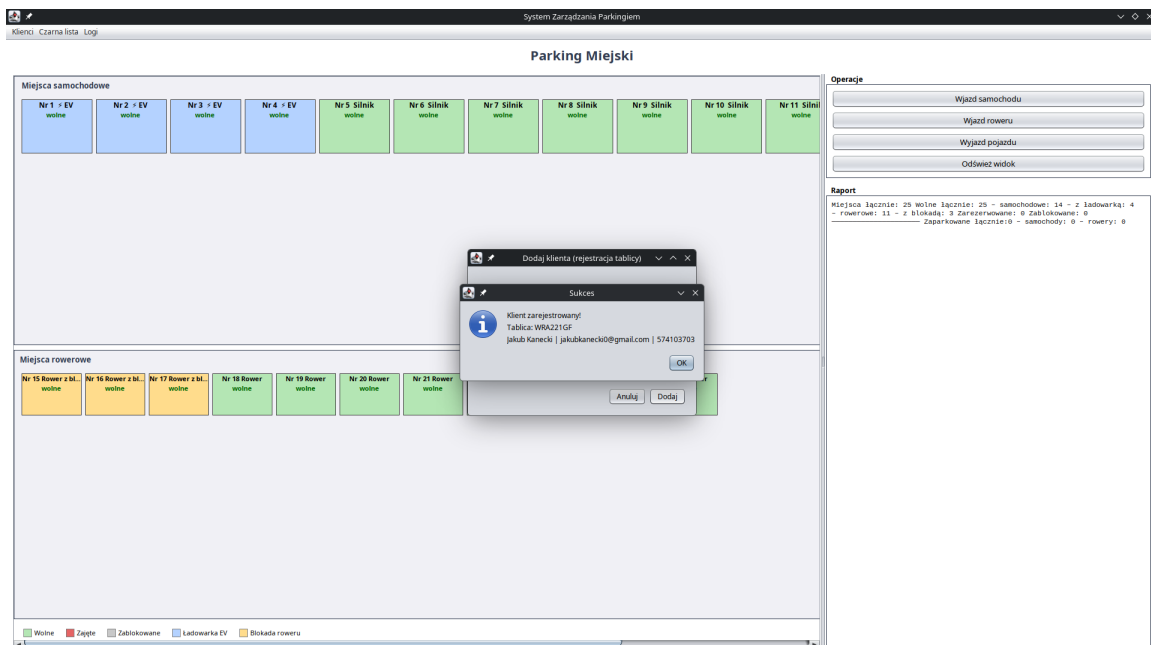
Rys. 10. To samo okno po wybraniu opcji „Rower” – etykieta pola zmienia się na „Nr kwitu”, a rower wyszukiwany jest po numerze kwitu.



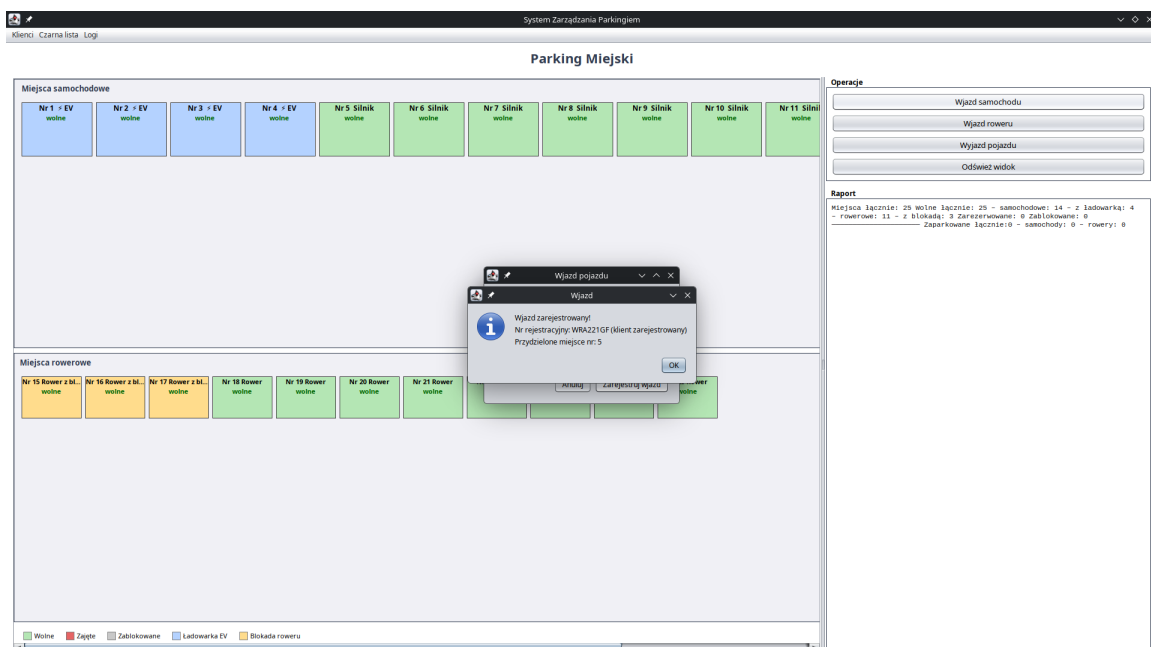
Rys. 11. Potwierdzenie wyjazdu roweru i zwolnienie miejsca (kafelki wraca do koloru „wolne”).



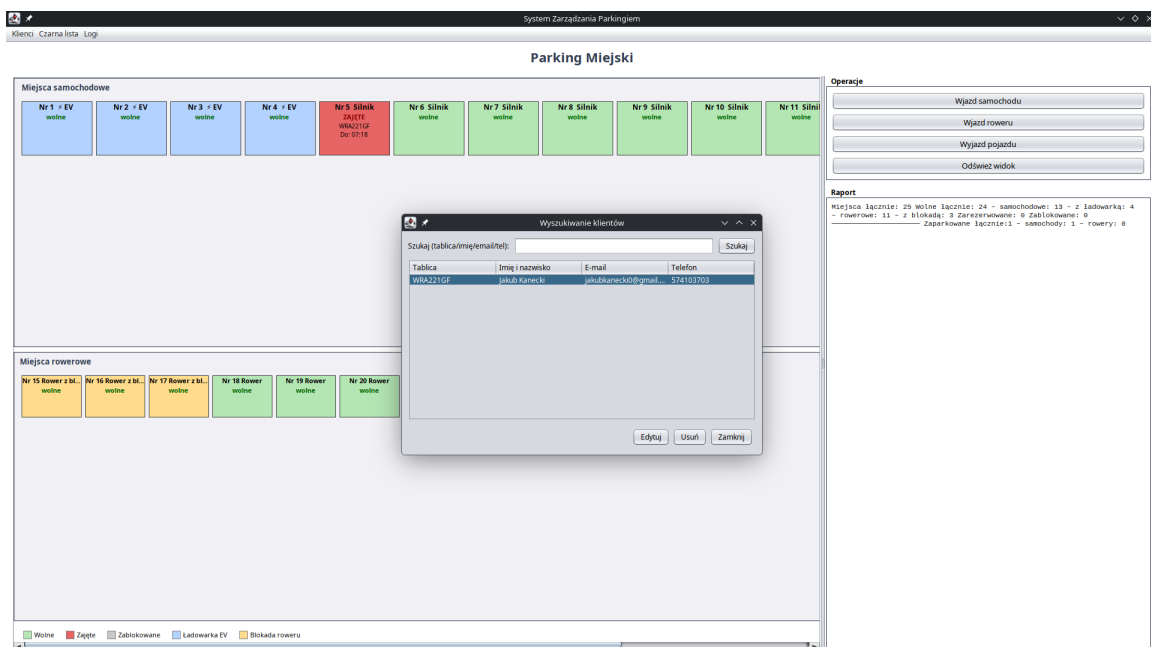
Rys. 12. Okno „Dodaj klienta” (menu Klienci) – rejestracja tablicy wraz z danymi właściciela; pola telefonu i e-maila są walidowane.



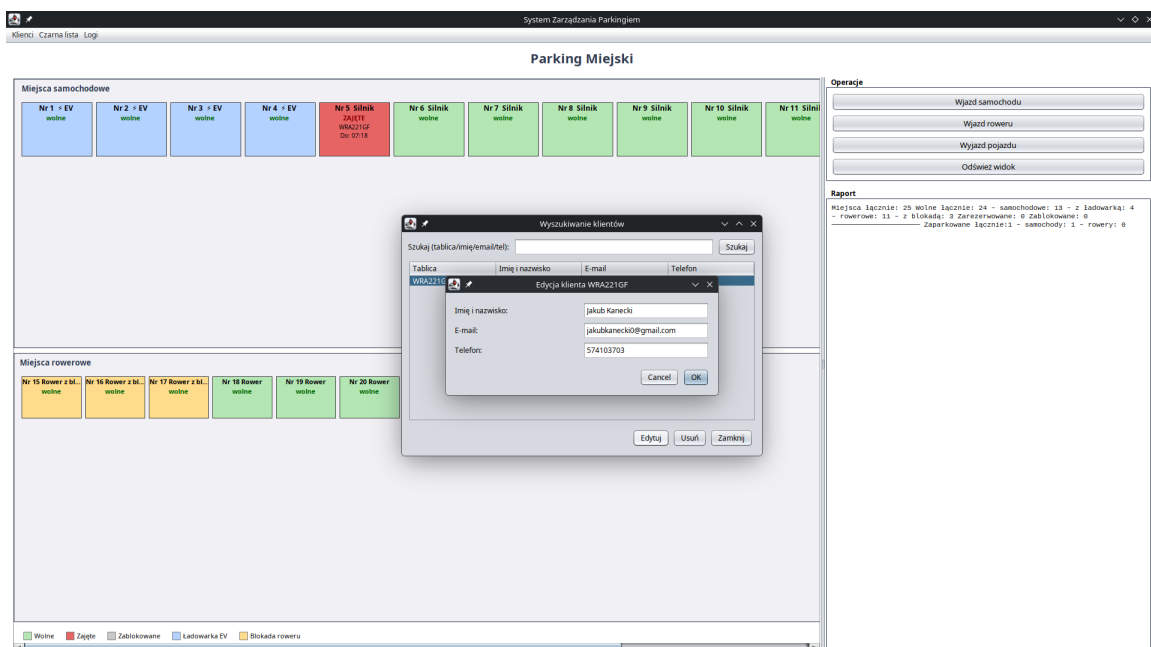
Rys. 13. Potwierdzenie pomyślnej rejestracji klienta.



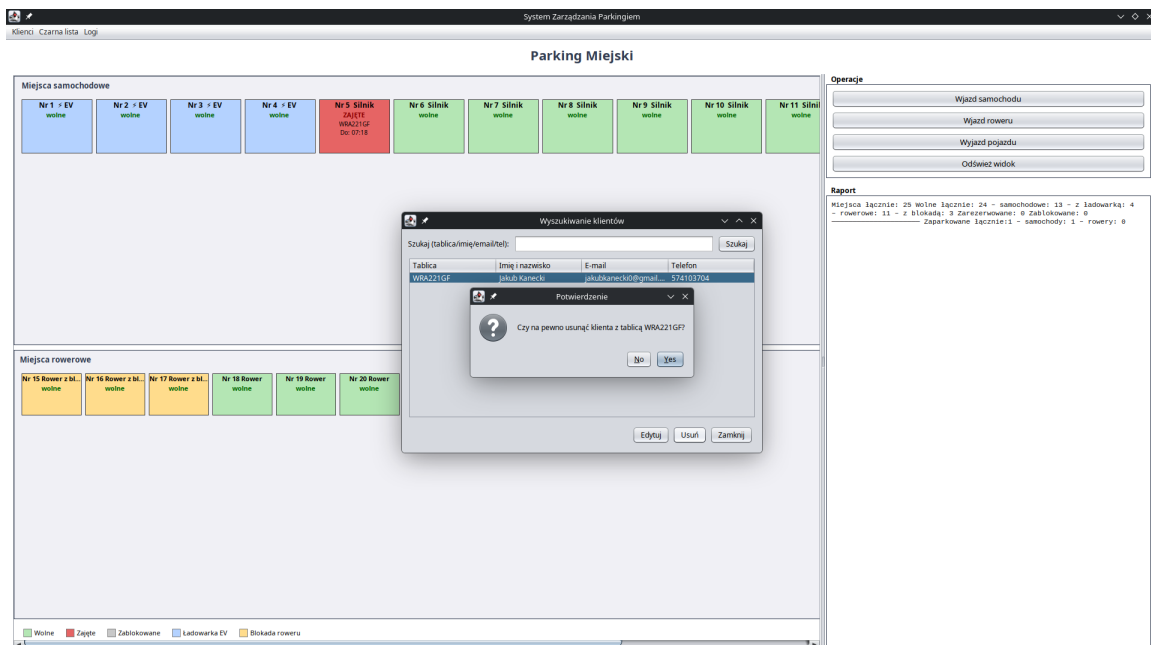
Rys. 14. Wyjazd pojazdu zarejestrowanego klienta – komunikat zawiera adnotację „(klient zarejestrowany)”.



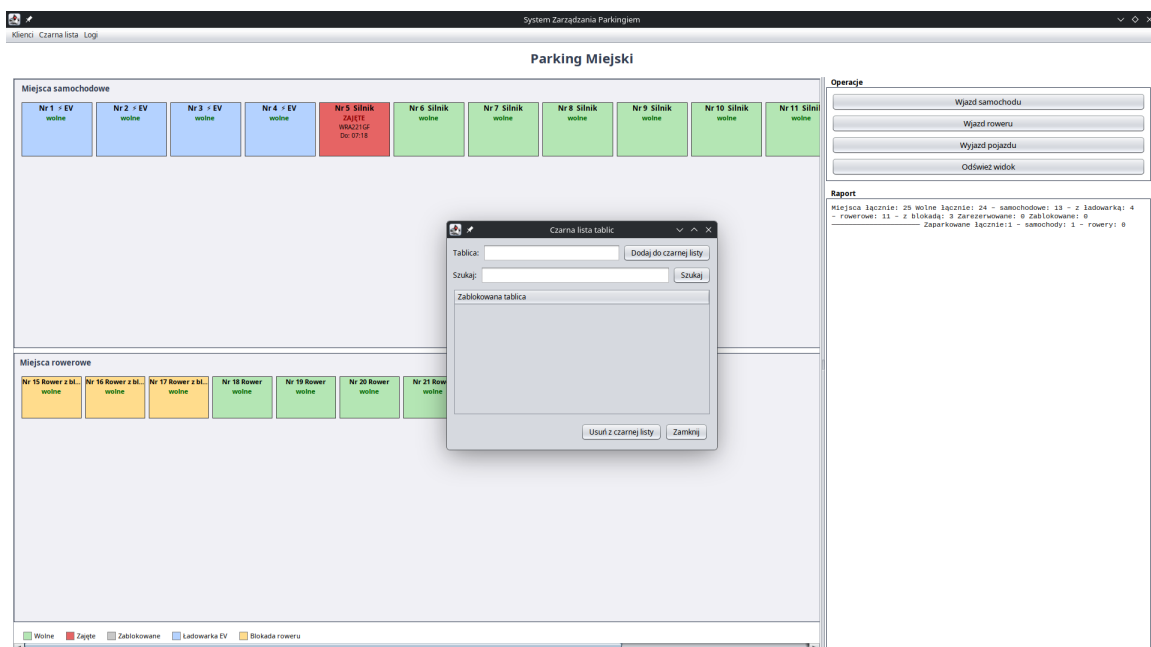
Rys. 15. Okno „Wyszukiwanie klientów” z tabelą i polem filtrowania po tablicy, imieniu, e-mailu lub telefonie.



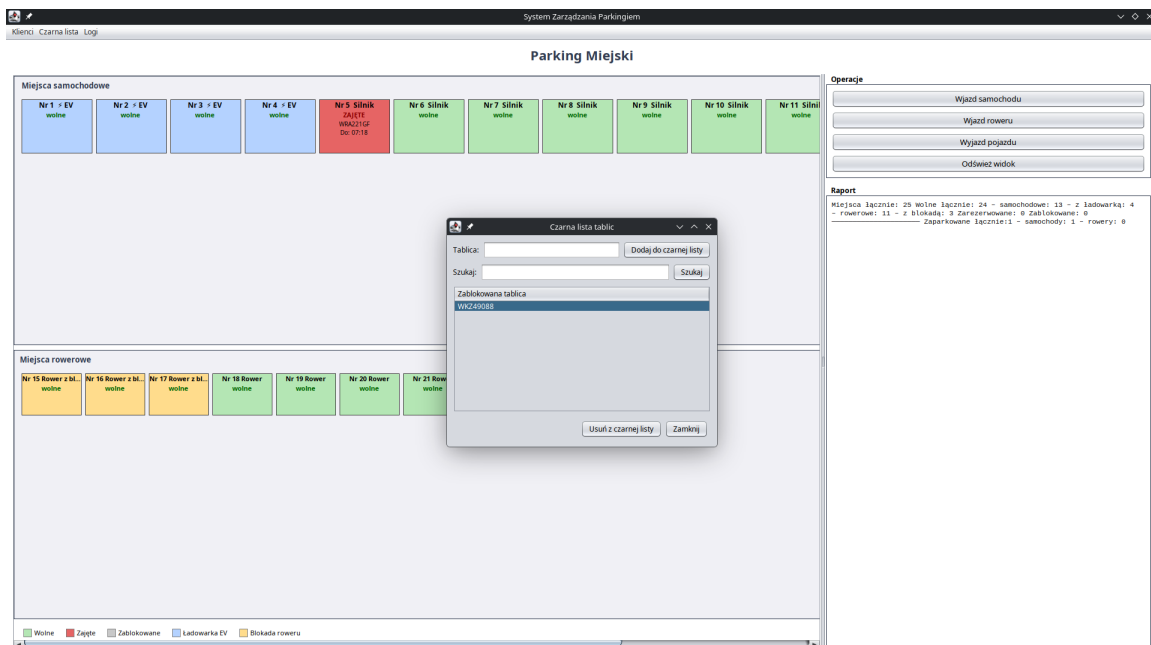
Rys. 16. Edycja danych wybranego klienta (przycisk „Edytuj”).



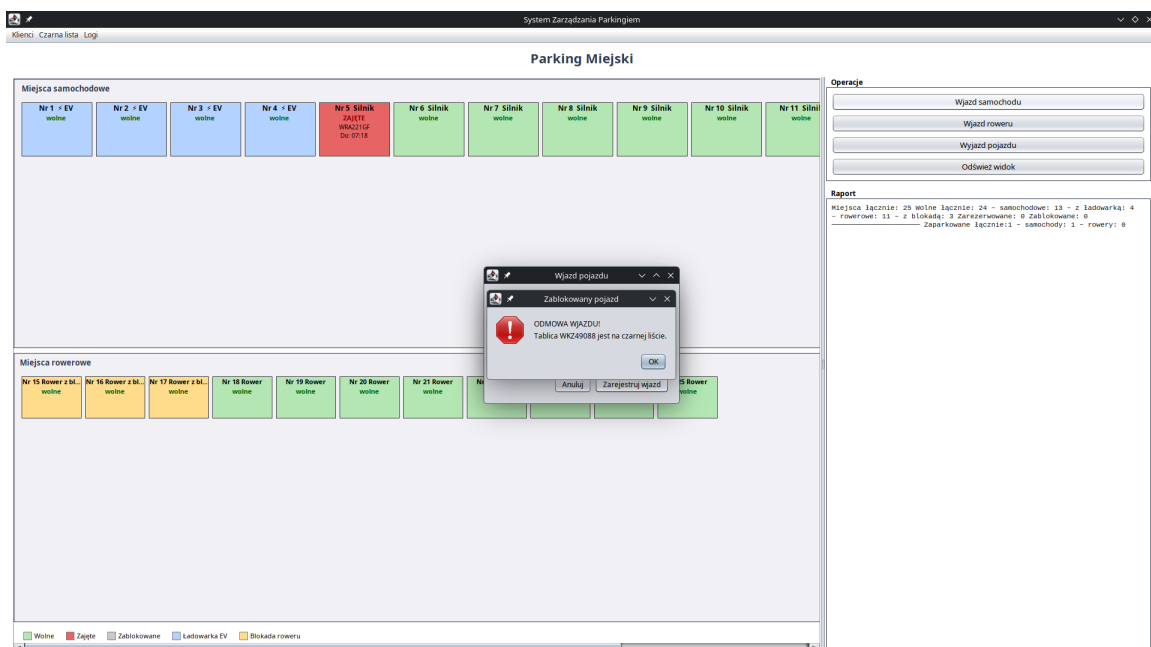
Rys. 17. Potwierdzenie usunięcia klienta z rejestru.



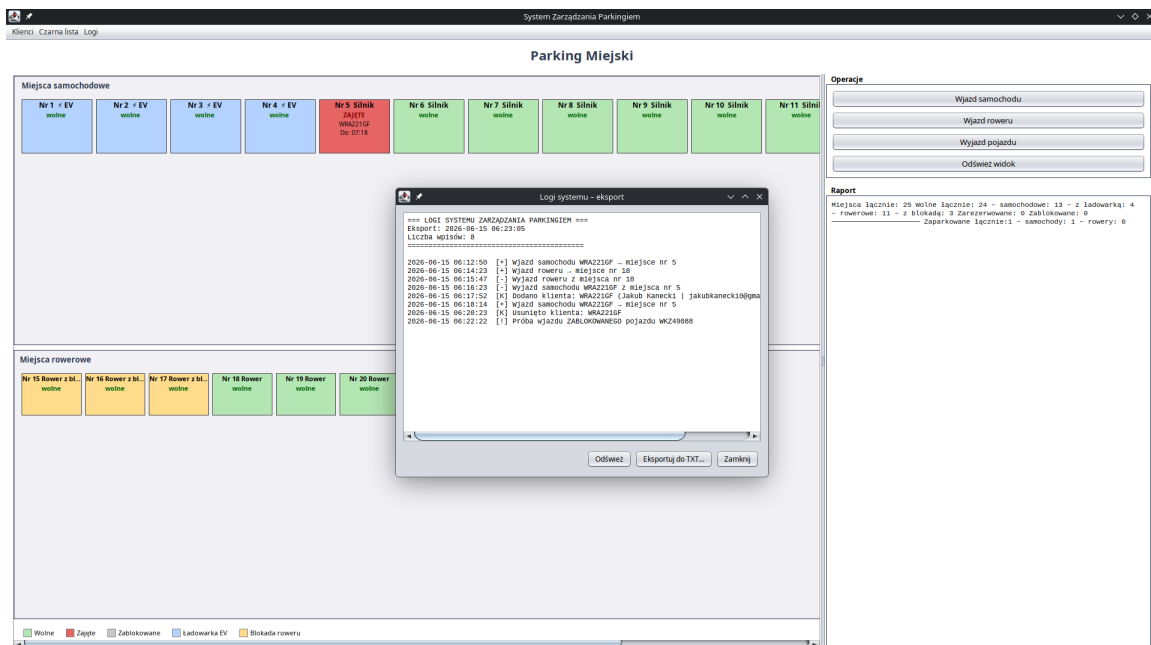
Rys. 18. Okno „Czarna lista tablic” (menu Czarna lista) – pole dodawania oraz wyszukiwarka.



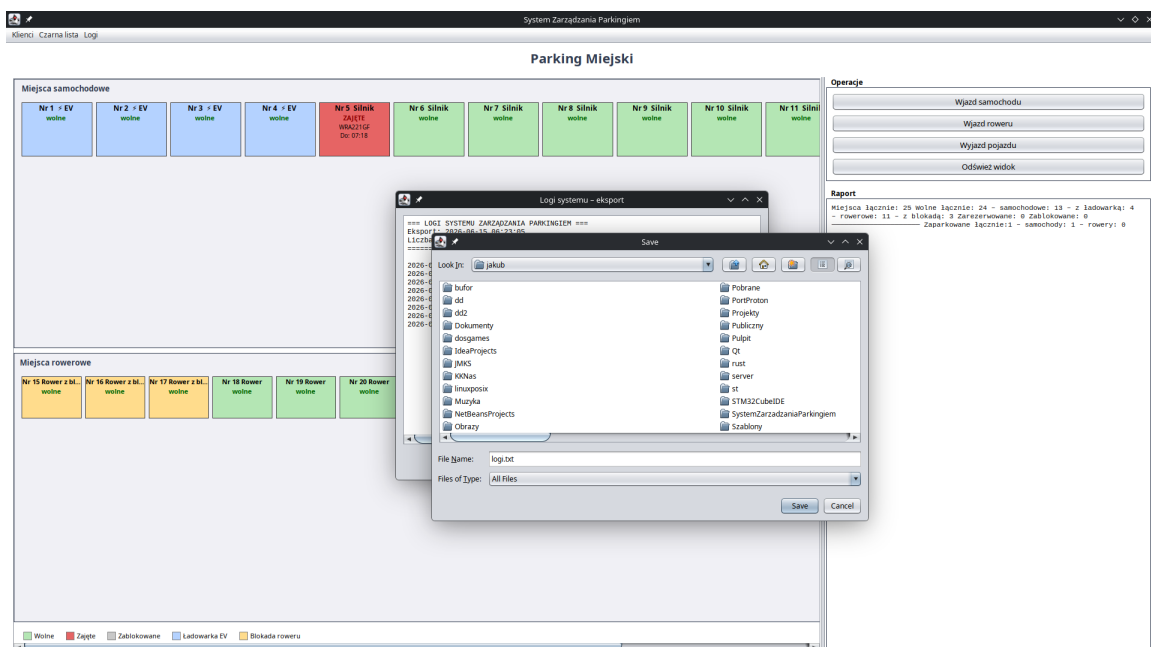
Rys. 19. Czarna lista po dodaniu zablokowanej tablicy.



Rys. 20. Próba wjazdu pojazdu znajdującego się na czarnej liście – system wyświetla komunikat „ODMOWA WJAZDU” i zapisuje zdarzenie w logach.



Rys. 21. Okno „Logi systemu – eksport” (menu Logi) z chronologiczną historią zdarzeń.



Rys. 22. Eksport logów do pliku TXT – okno wyboru lokalizacji zapisu.

6) Opis GUI programu*

Interfejs graficzny zbudowano w bibliotece Swing z zastosowaniem wyglądu Nimbus. Całość obsługiwana jest w wątku zdarzeń (EDT). Okno główne (MainWindow) wykorzystuje menedżery układu BorderLayout, GridLayout oraz autorski WrapLayout (zawijanie kafelków miejsc przy zmianie szerokości okna). Struktura GUI jest następująca:

- Pasek menu: Klienci (dodawanie, wyszukiwanie/edycja), Czarna lista (zarządzanie), Logi (podgląd i eksport).
- Sekcja centralna (JSplitPane): po lewej wizualizacja miejsc w dwóch grupach (samochodowe i rowerowe) jako kolorowe kafelki, po prawej panel boczny.
- Panel „Operacje”: przyciski Wjazd samochodu, Wjazd roweru, Wyjazd pojazdu, Odśwież widok.
- Panel „Raport”: bieżące statystyki (liczba miejsc, wolne wg typu, zarezerwowane, zablokowane, zaparkowane pojazdy) generowane przez klasę Raporty.
- Legenda kolorów: wolne (zielony), zajęte (czerwony), zablokowane (szary), ładowarka EV (niebieski), blokada roweru (pomarańczowy).
- Okna dialogowe (JDialog) są modalne; po zatwierdzeniu operacji widok główny jest automatycznie odświeżany metodą odswiezWidok().

Każdy kafelek miejsca prezentuje numer, typ (np. ładowarka EV / rower z blokadą), status oraz – dla miejsc zajętych – tablicę pojazdu i planowaną godzinę wyjazdu lub numer kwitu. Dzięki temu stan parkingu jest czytelny na pierwszy rzut oka.

7) Wnioski i Uwagi

// dotyczące realizacji własnego programu. Tutaj należy napisać co udało się państwu oprogramować, oraz czego zabrakło a chcielibyście jeszcze zaimplementować.

Zaimplementowano:

- obsługę dwóch typów pojazdów (samochód, rower) i dwóch typów miejsc, z wykorzystaniem dziedziczenia, polimorfizmu, klas abstrakcyjnych i interfejsu (IRaporty);
- automatyczny przydział miejsc z priorytetem ładowarek dla pojazdów elektrycznych oraz mechanizmem awaryjnym;
- rejestrację wjazdu i wyjazdu (samochody po tablicy, rowery po numerze kwitu) wraz z sesjami postoju i pomiarem czasu;
- rejestr klientów (tablica → właściciel) z wyszukiwaniem, edycją i usuwaniem oraz czarną listę tablic blokującą wjazd;
- dziennik zdarzeń (logi) z podglądem i eksportem do pliku TXT;
- panel raportowy z bieżącymi statystykami oraz czytelną, kolorystyczną wizualizację stanu parkingu.

Czego zabrakło / możliwe rozszerzenia:

- naliczania opłat za postój na podstawie zmierzonego czasu sesji;
- obsługi rezerwacji miejsc oraz ręcznego blokowania/odblokowywania pojedynczych miejsc z poziomu GUI;

8) Listing kodu C++ / Java – wraz z komentarzami.

//zawartość plików źródłowych – tylko kod własny

Poniżej zamieszczono pełny kod źródłowy własnych klas (wraz z komentarzami), pogrupowany według pakietów.

Pakiet com.systemzarzadzaniaparkingiem.model

STATUS_MIEJSCA.java

```
package com.systemzarzadzaniaparkingiem.model;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public enum STATUS_MIEJSCA { // zrobione
    DOSTEPNE, ZAJETE, ZABLOKOWANE, ZAREZERWOWANE
}
```

Pojazd.java

```
package com.systemzarzadzaniaparkingiem.model;
import java.util.UUID;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public abstract class Pojazd {
    protected UUID uuid;
    public Pojazd() { uuid = UUID.randomUUID(); }
}
```

Samochod.java

```
package com.systemzarzadzaniaparkingiem.model;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Samochod extends Pojazd {
    private String tablica;
    private boolean elektryczny;

    public Samochod(String tablica, boolean elektryczny) {
        super();
        this.tablica = tablica;
        this.elektryczny = elektryczny;
    }

    public Samochod(String tablica) { this(tablica, false); }

    public boolean PorownajTablice(String tablica2) { return
this.tablica.equalsIgnoreCase(tablica2); }
    public boolean CzyElektryczny() { return elektryczny; }
    public String GetTablica() { return tablica; }
}
```

Rower.java

```
package com.systemzarzadzaniaparkingiem.model;
```

```

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Rower extends Pojazd {
    private boolean elektryczny;

    public Rower(boolean elektryczny) {
        super();
        this.elektryczny = elektryczny;
    }

    public Rower() { this(false); }
    public boolean CzyElektryczny() { return elektryczny; }
}

```

Wlasciciel.java

```

package com.systemzarzadzaniaparkingiem.model;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Wlasciciel {
    private String imieNazwisko;
    private String eMail;
    private String nrTelefonu;

    public Wlasciciel(String imieNazwisko, String eMail, String nrTelefonu) {
        this.imieNazwisko = imieNazwisko;
        this.eMail = eMail;
        this.nrTelefonu = nrTelefonu;
    }

    public Wlasciciel() { this("Jan Kowalski", "example@example.com", "123456789"); }

    // Odczyt danych - konieczne do wyświetlania w GUI i wyszukiwania
    public String GetImieNazwisko() { return imieNazwisko; }
    public String GeteMail() { return eMail; }
    public String GetNrTelefonu() { return nrTelefonu; }

    public void Aktualizuj(String imieNazwisko, String eMail, String nrTelefonu) {
        this.imieNazwisko = imieNazwisko;
        this.eMail = eMail;
        this.nrTelefonu = nrTelefonu;
    }

    public String Dane() {
        return imieNazwisko + " | " + eMail + " | " + nrTelefonu;
    }
}

```

SesjaParkingowa.java

```

package com.systemzarzadzaniaparkingiem.model;

import java.time.Duration;
import java.time.Instant;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski

```

```

*/
public class SesjaParkingowa {
    public Pojazd pojazd;
    private Instant czasWjazdu;
    private Instant zaplanowanyCzasWyjazdu;
    private Instant rzeczywistyCzasWyjazdu;
    private int numerKwitu; // dla rowerów

    public SesjaParkingowa(Pojazd pojazd, Instant czasWjazdu, Instant zaplanowanyCzasWyjazdu) {
        this.pojazd = pojazd;
        this.czasWjazdu = czasWjazdu;
        this.zaplanowanyCzasWyjazdu = zaplanowanyCzasWyjazdu;
        this.numerKwitu = -1;
    }

    public SesjaParkingowa(Pojazd pojazd, Instant czasWjazdu, Instant zaplanowanyCzasWyjazdu, int
numerKwitu) {
        this(pojazd, czasWjazdu, zaplanowanyCzasWyjazdu);
        this.numerKwitu = numerKwitu;
    }

    public Duration ObliczZaplanowanyCzas() {
        return Duration.between(this.czasWjazdu, this.zaplanowanyCzasWyjazdu);
    }

    public Duration ObliczRzeczywistyCzas() {
        if (rzeczywistyCzasWyjazdu == null) return Duration.between(this.czasWjazdu,
Instant.now());
        return Duration.between(this.czasWjazdu, this.rzeczywistyCzasWyjazdu);
    }

    public void Zamknij() { this.rzeczywistyCzasWyjazdu = Instant.now(); }

    public boolean CzyWyjazdPunktualny() {
        if (rzeczywistyCzasWyjazdu == null) return false;
        return this.zaplanowanyCzasWyjazdu.isAfter(this.rzeczywistyCzasWyjazdu);
    }

    public Instant getCzasWjazdu() { return czasWjazdu; }
    public Instant getZaplanowanyCzasWyjazdu() { return zaplanowanyCzasWyjazdu; }
    public int getNumerKwitu() { return numerKwitu; }
}

```

MiejsceParkingowe.java

```

package com.systemzarzadzaniaparkingiem.model;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public abstract class MiejsceParkingowe {
    protected static int iloscMiejsc = 0;
    protected int nrMiejsc;
    protected STATUS_MIEJSCA status;
    protected SesjaParkingowa aktywnaSesja;

    public MiejsceParkingowe() {
        this.iloscMiejsc++;
        this.nrMiejsc = this.iloscMiejsc;
        this.status = STATUS_MIEJSCA.DOSTEPNE;
        this.aktywnaSesja = null;
    }
}

```



```

    public static void resetLicznik() { iloscMiejsc = 0; }

    public void ZablockujMiejsce() { this.status = STATUS_MIEJSCA.ZABLOKOWANE; }
    public void ZarezerwujMiejsce() { this.status = STATUS_MIEJSCA.ZAREZERWOWANE; }
    public void OdblockujMiejsce() { this.status = STATUS_MIEJSCA.DOSTEPNE; }

    public void ZajmijMiejsce(SesjaParkingowa sesja) {
        this.aktywnaSesja = sesja;
        this.status = STATUS_MIEJSCA.ZAJETE;
    }

    public void ZwolnijMiejsce() {
        this.aktywnaSesja = null;
        this.OdblockujMiejsce();
    }

    public boolean CzyDostepne() { return this.status == STATUS_MIEJSCA.DOSTEPNE; }
    public boolean CzyCzynne() { return this.status != STATUS_MIEJSCA.ZABLOKOWANE; }
    public boolean CzyZarezerwowane() { return this.status == STATUS_MIEJSCA.ZAREZERWOWANE; }
    public boolean CzyZajete() { return this.status == STATUS_MIEJSCA.ZAJETE; }
    public boolean CzyPosiadaLadowarke() { return false; }
    public boolean CzyPosiadaBlokade() { return false; }
    public boolean PorownajId(int id) { return this.nrMiejsc == id; }

    public int getNrMiejsc() { return nrMiejsc; }
    public STATUS_MIEJSCA getStatus() { return status; }
    public SesjaParkingowa getAktywnaSesja() { return aktywnaSesja; }
}

```

MiejsceSamochodowe.java

```

package com.systemzarzadzaniaparkingiem.model;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class MiejsceSamochodowe extends MiejsceParkingowe {
    private boolean ladowarka;

    public MiejsceSamochodowe(boolean ladowarka) { super(); this.ladowarka = ladowarka; }
    public MiejsceSamochodowe() { super(); this.ladowarka = false; }

    @Override public boolean CzyPosiadaLadowarke() { return this.ladowarka; }
}

```

MiejsceRowerowe.java

```

package com.systemzarzadzaniaparkingiem.model;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class MiejsceRowerowe extends MiejsceParkingowe {
    private boolean blokada;

    public MiejsceRowerowe(boolean blokada) { super(); this.blokada = blokada; }
    public MiejsceRowerowe() { super(); this.blokada = false; }

    @Override public boolean CzyPosiadaBlokade() { return this.blokada; }
}

```

```
}
```

ZarejestrowaneTablice.java

```
package com.systemzarzadzaniaparkingiem.model;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Set;
import java.util.Map;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class ZarejestrowaneTablice {
    private HashSet<String> tablice;
    private HashMap<String, Wlasciciel> wlasciciele;
    private HashSet<String> zablokowaneTablice;

    public ZarejestrowaneTablice() {
        this.wlasciciele = new HashMap<>();
        this.zablokowaneTablice = new HashSet<>();
        this.tablice = new HashSet<>();
    }

    public void DodajTablice(String tablica, Wlasciciel wlasciciel) {
        this.wlasciciele.put(tablica.toUpperCase(), wlasciciel);
        this.tablice.add(tablica.toUpperCase());
    }

    public void UsunTablice(String tablica) {
        this.wlasciciele.remove(tablica.toUpperCase());
        this.tablice.remove(tablica.toUpperCase());
    }

    public void ZablokujTablice(String tablica) {
        this.zablokowaneTablice.add(tablica.toUpperCase());
        this.tablice.remove(tablica.toUpperCase());
    }

    public void OdblokujTablice(String tablica) {
        this.zablokowaneTablice.remove(tablica.toUpperCase());
        this.tablice.add(tablica.toUpperCase());
    }

    public boolean CzyZarejestrowana(String tablica) {
        return this.tablice.contains(tablica);
    }

    public boolean CzyZablokowana(String tablica) {
        return this.zablokowaneTablice.contains(tablica.toUpperCase());
    }

    public Wlasciciel Wlasciciel(String tablica) {
        return this.wlasciciele.get(tablica.toUpperCase());
    }

    public Map<String, Wlasciciel> GetWszystkieTablice() { return wlasciciele; }
    public Set<String> GetZablokowaneTablice() { return zablokowaneTablice; }
}
```

Logi.java

```

package com.systemzarzadzaniaparkingiem.model;

import java.time.Instant;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Logi {
    private ArrayList<String> logi;
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss").withZone(ZoneId.systemDefault());

    public Logi() { this.logi = new ArrayList<>(); }

    private String Czas() { return FMT.format(Instant.now()); }

    public void WjazdSamochodu(String tab, int miejsce) {
        logi.add(Czas() + " [+] Wjazd samochodu " + tab + " → miejsce nr " + miejsce);
    }

    public void WjazdRoweru(int miejsce) {
        logi.add(Czas() + " [+] Wjazd roweru → miejsce nr " + miejsce);
    }

    public void WyjazdSamochodu(String tab, int miejsce) {
        logi.add(Czas() + " [-] Wyjazd samochodu " + tab + " z miejsca nr " + miejsce);
    }

    public void WyjazdRoweru(int miejsce) {
        logi.add(Czas() + " [-] Wyjazd roweru z miejsca nr " + miejsce);
    }

    public void ProbaWjazduZablokowanegoPojazdu(String tab) {
        logi.add(Czas() + " [!] Próba wjazdu ZABLOKOWANEGO pojazdu " + tab);
    }

    public void DodanoKlienta(String tab, String dane) {
        logi.add(Czas() + " [K] Dodano klienta: " + tab + " (" + dane + ")");
    }

    public void UsunietKlient(String tab) {
        logi.add(Czas() + " [K] Usunięto klienta: " + tab);
    }

    public void ZablokowanaTablica(String tab) {
        logi.add(Czas() + " [B] Dodano do czarnej listy: " + tab);
    }

    public void OdblokowanaTablica(String tab) {
        logi.add(Czas() + " [B] Usunięto z czarnej listy: " + tab);
    }

    public void EksportLogow(String sciezka) {
        logi.add(Czas() + " [E] Eksport logów do pliku: " + sciezka);
    }

    public List<String> GetLogi() { return Collections.unmodifiableList(logi); }
}

```

```

    public String Serializuj() {
        StringBuilder sb = new StringBuilder();
        sb.append("=== LOGI SYSTEMU ZARZĄDZANIA PARKINGIEM ===\n");
        sb.append("Eksport: ").append(Czas()).append("\n");
        sb.append("Liczba wpisów: ").append(logi.size()).append("\n");
        sb.append("=====\n\n");
        for (String log : logi) {
            sb.append(log).append("\n");
        }
        return sb.toString();
    }
}

```

IRaporty.java

```

package com.systemzarzadzaniaparkingiem.model;

/**
 * @author Jakub Kanecki, Dawid Lazarski
 */
public interface IRaporty { // do zmian w UML
    // miejsca
    public int IloscMiejsc();
    public int IloscWolnychMiejsc();
    public int IloscWolnychMiejscSamoch();
    public int IloscWolnychMiejscLadowarka();
    public int IloscWolnychMiejscRower();
    public int IloscWolnychMiejscBlokada();
    public int IloscMiejscZarezerwowanych();
    public int IloscMiejscZablokowanych();

    // pojazdy
    // IloscZaparkowanychPojazdow do wyliczenia przez IloscMiejsc() - IloscWolnychMiejsc()
    public int IloscZaparkowanychSamoch();
    public int IloscZaparkowanychRower();
    // raport tekstowy
}

```

Raporty.java

```

package com.systemzarzadzaniaparkingiem.model;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Raporty implements IRaporty {
    Parking parking;

    Raporty(Parking parking) {
        this.parking = parking;
    }
    @Override
    public int IloscMiejsc() {
        return parking.miejscaParkingowe.size();
    }
    @Override
    public int IloscWolnychMiejsc() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce.CzyDostepne())
                count++;
    }
}

```

```

        return count;
    }
    @Override
    public int IloscWolnychMiejscSamoch() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceSamochodowe)
                count++;

        return count;
    }
    @Override
    public int IloscWolnychMiejscLadowarka() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceSamochodowe)
                if (miejsce.CzyPosiadaLadowarke())
                    count++;

        return count;
    }
    @Override
    public int IloscWolnychMiejscRower() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceRowerowe)
                count++;

        return count;
    }
    @Override
    public int IloscWolnychMiejscBlokada() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceRowerowe)
                if (miejsce.CzyPosiadaBlokade())
                    count++;

        return count;
    }
    @Override
    public int IloscMiejscZarezerwowanych() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce.CzyZarezerwowane())
                count++;

        return count;
    }
    @Override
    public int IloscMiejscZablokowanych() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (!miejsce.CzyCzynne())
                count++;

        return count;
    }
    @Override
    public int IloscZaparkowanychSamoch() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce instanceof MiejsceSamochodowe && miejsce.CzyZajete())
                count++;

        return count;
    }

```

```

    }
    @Override
    public int IloscZaparkowanychRower() {
        int count = 0;
        for (MiejsceParkingowe miejsce : parking.miejscaParkingowe)
            if (miejsce instanceof MiejsceRowerowe && miejsce.CzyZajete())
                count++;

        return count;
    }

    public String GenerujRaportTekstowy() {

        String szablon = ""
        Miejsca:
        - łącznie: %d
        - dostępne:
            - łącznie: %d
            - samochodowe: %d
            - w tym z ładowarką: %d
            - rowerowe: %d
            - w tym z blokadą: %d
            - zarezerwowane: %d
            - zablokowane: %d

        Pojazdy:<br>
        - zaparkowane łącznie: %d
        - samochody: %d
        - rowery: %d

        """;

        return String.format(szablon,
            IloscMiejsc(),
            IloscWolnychMiejsc(),
            IloscWolnychMiejscSamoch(),
            IloscWolnychMiejscLadowarka(),
            IloscWolnychMiejscRower(),
            IloscWolnychMiejscBlokada(),
            IloscMiejscZarezerwowanych(),
            IloscMiejscZablokowanych(),
            IloscMiejsc() - IloscWolnychMiejsc(),
            IloscZaparkowanychSamoch(),
            IloscZaparkowanychRower()
        );
    }
}

```

Parking.java

```

package com.systemzarzadzaniaparkingiem.model;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class Parking {
    private String nazwa;
    public ZarejestrowaneTablice zarejestrowaneTablice;
    public Raporty raporty;
    public ArrayList<MiejsceParkingowe> miejscaParkingowe;
}

```

```

public Logi logi;

public Parking(String nazwa) {
    this.nazwa = nazwa;
    this.zarejestrowaneTablice = new ZarejestrowaneTablice();
    this.raporty = new Raporty(this);
    this.miejscaParkingowe = new ArrayList<>();
    this.logi = new Logi();
}

public Parking() {
    this("Parking Miejski");
    DodajMiejsca();
}

private void DodajMiejsca() {
    for (int i = 0; i < 4; i++)
        this.miejscaParkingowe.add(new MiejsceSamochodowe(true));
    for (int i = 0; i < 10; i++)
        this.miejscaParkingowe.add(new MiejsceSamochodowe());
    for (int i = 0; i < 3; i++)
        this.miejscaParkingowe.add(new MiejsceRowerowe(true));
    for (int i = 0; i < 8; i++)
        this.miejscaParkingowe.add(new MiejsceRowerowe());
}

public String GetNazwa() { return nazwa; }

// — operacje parkingowe —————

public void ZarejestrujWjazd(Pojazd pojazd, MiejsceParkingowe miejsce, Instant
zakonczeniePostoju) {
    miejsce.ZajmijMiejsce(new SesjaParkingowa(pojazd, Instant.now(), zakonczeniePostoju));
    if (pojazd instanceof Samochod) {
        logi.WjazdSamochodu(((Samochod) pojazd).GetTablica(), miejsce.getNrMiejsca());
    } else if (pojazd instanceof Rower) {
        logi.WjazdRoweru(miejsce.getNrMiejsca());
    }
}

public void ZarejestrujWyjazd(MiejsceParkingowe miejsce) {
    if (miejsce != null && miejsce.CzyZajete()) {
        Pojazd pojazd = miejsce.getAktywnaSesja() != null ?
miejsce.getAktywnaSesja().pojazd : null;
        miejsce.getAktywnaSesja().Zamknij();
        miejsce.ZwolnijMiejsce();
        if (pojazd instanceof Samochod) {
            logi.WyjazdSamochodu(((Samochod) pojazd).GetTablica(), miejsce.getNrMiejsca());
        } else if (pojazd instanceof Rower) {
            logi.WyjazdRoweru(miejsce.getNrMiejsca());
        }
    }
}

public MiejsceParkingowe ZnajdzMiejsce(Samochod pojazd) {
    if (pojazd.CzyElektryczny()) {
        for (MiejsceParkingowe miejsce : this.miejscaParkingowe) {
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceSamochodowe &&
miejsce.CzyPosiadaLadowarke())
                return miejsce;
        }
    } else {
        for (MiejsceParkingowe miejsce : this.miejscaParkingowe) {
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceSamochodowe && !
miejsce.CzyPosiadaLadowarke())
                return miejsce;
        }
    }
}

```

```

    }
    for (MiejsceParkingowe miejsce : this.miejscaParkingowe) {
        if (miejsce.CzyDostepne() && miejsce instanceof MiejsceSamochodowe)
            return miejsce;
    }
    return null;
}

public MiejsceParkingowe ZnajdzMiejsce(Rower pojazd, boolean wlasnaBlokada) {
    if (!wlasnaBlokada) {
        for (MiejsceParkingowe miejsce : this.miejscaParkingowe) {
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceRowerowe &&
miejsce.CzyPosiadaBlokade())
                return miejsce;
        }
    } else {
        for (MiejsceParkingowe miejsce : this.miejscaParkingowe) {
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceRowerowe && !
miejsce.CzyPosiadaBlokade())
                return miejsce;
        }
        for (MiejsceParkingowe miejsce : this.miejscaParkingowe) {
            if (miejsce.CzyDostepne() && miejsce instanceof MiejsceRowerowe)
                return miejsce;
        }
    }
    return null;
}

/** Zwraca -1 jeśli zablokowana, 1 jeśli zarejestrowana, 0 jeśli nieznana */
public int sprawdzTabliceWjazd(String tablica) {
    if (this.zarejestrowaneTablice.CzyZablokowana(tablica)) return -1;
    if (this.zarejestrowaneTablice.CzyZarejestrowana(tablica)) return 1;
    return 0;
}

public MiejsceParkingowe WyszukajMiejsce(int id) {
    for (MiejsceParkingowe miejsce : this.miejscaParkingowe) {
        if (miejsce.PorownajId(id)) return miejsce;
    }
    return null;
}

public boolean CzyIstniejeSesjaNaMiejscuNr(int id) {
    MiejsceParkingowe miejsce = WyszukajMiejsce(id);
    return miejsce != null && miejsce.CzyZajete();
}

public List<MiejsceParkingowe> GetMiejscaSamochodowe() {
    List<MiejsceParkingowe> lista = new ArrayList<>();
    for (MiejsceParkingowe m : miejscaParkingowe)
        if (m instanceof MiejsceSamochodowe) lista.add(m);
    return lista;
}

public List<MiejsceParkingowe> GetMiejscaRowerowe() {
    List<MiejsceParkingowe> lista = new ArrayList<>();
    for (MiejsceParkingowe m : miejscaParkingowe)
        if (m instanceof MiejsceRowerowe) lista.add(m);
    return lista;
}

public MiejsceParkingowe ZnajdzMiejscePojazdaZTabl(String tablica) {
    for (MiejsceParkingowe m : miejscaParkingowe) {
        if (m.CzyZajete() && m.getAktywnaSesja() != null) {
            Pojazd p = m.getAktywnaSesja().pojazd;
            if (p instanceof Samochod && ((Samochod)

```



```

p).GetTablica().equalsIgnoreCase(tablica))
        return m;
    }
    return null;
}

public void DodajKlienta(String tablica, Wlasciciel wlasciciel) {
    zarejestrowaneTablice.DodajTablice(tablica, wlasciciel);
    logi.DodanoKlienta(tablica, wlasciciel.Dane());
}

public void UsunKlienta(String tablica) {
    zarejestrowaneTablice.UsunTablice(tablica);
    logi.UsunietKlient(tablica);
}

public Wlasciciel ZnajdzWlasciciela(String tablica) {
    return zarejestrowaneTablice.Wlasciciel(tablica);
}
}

```

Pakiet com.systemzarzadzaniaparkingiem.widok

MainWindow.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class MainWindow extends JFrame {

    private final Parking parking;
    private JPanel panelSamochodowe;
    private JPanel panelRowerowe;
    private JLabel labelRaport;
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("HH:mm").withZone(ZoneId.systemDefault());

    // Kolory
    private static final Color CLR_WOLNE = new Color(180, 230, 180);
    private static final Color CLR_ZAJETE = new Color(230, 100, 100);
    private static final Color CLR_ZABLOK = new Color(200, 200, 200);
    private static final Color CLR_LADOW = new Color(180, 210, 255);
    private static final Color CLR_ROWER_BL = new Color(255, 220, 140);
    private static final Color CLR_TLO_SEKCJI = new Color(240, 240, 245);
    private static final Color CLR_NAGLOWEK = new Color(50, 60, 80);

    public MainWindow() {
        this.parking = new Parking();
        buildUI();
        odswiezWidok();
        setTitle("System Zarządzania Parkingiem");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
}

```

```

        setMinimumSize(new Dimension(900, 600));
        setMaximumSize(new Dimension(1280, 720));
        pack();
        setSize(900, 600);
        setLocationRelativeTo(null);
    }

    private void buildUI() {
        // Pasek menu
        JMenuBar menuBar = new JMenuBar();

        JMenu menuKlienci = new JMenu("Klienci");
        JMenuItem miDodajKlienta = new JMenuItem("Dodaj klienta...");
        JMenuItem miWyszukajKlienta = new JMenuItem("Wyszukaj / edytuj klientów...");
        menuKlienci.add(miDodajKlienta);
        menuKlienci.add(miWyszukajKlienta);

        JMenu menuCzarnaLista = new JMenu("Czarna lista");
        JMenuItem miCzarnaLista = new JMenuItem("Zarządzaj czarną listą...");
        menuCzarnaLista.add(miCzarnaLista);

        JMenu menuLogi = new JMenu("Logi");
        JMenuItem miEksport = new JMenuItem("Podgląd i eksport logów...");
        menuLogi.add(miEksport);

        menuBar.add(menuKlienci);
        menuBar.add(menuCzarnaLista);
        menuBar.add(menuLogi);
        setJMenuBar(menuBar);

        miDodajKlienta.addActionListener(e -> { new DialogDodajKlienta(this,
parking).setVisible(true); odswiezWidok(); });
        miWyszukajKlienta.addActionListener(e -> { new DialogWyszukajKlienta(this,
parking).setVisible(true); odswiezWidok(); });
        miCzarnaLista.addActionListener(e -> { new DialogCzarnaLista(this,
parking).setVisible(true); odswiezWidok(); });
        miEksport.addActionListener(e -> new DialogEksportLogow(this, parking).setVisible(true));

        // Główny layout
        JPanel root = new JPanel(new BorderLayout(10, 10));
        root.setBorder(new EmptyBorder(10, 10, 10, 10));
        root.setBackground(Color.WHITE);
        setContentPane(root);

        // Nagłówek
        JLabel title = new JLabel(parking.GetNazwa(), JLabel.CENTER);
        title.setFont(new Font("SansSerif", Font.BOLD, 22));
        title.setForeground(CLR_NAGLOWEK);
        title.setBorder(new EmptyBorder(0, 0, 6, 0));
        root.add(title, BorderLayout.NORTH);

        // Środek: widok + boczny panel
        JSplitPane split = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT);
        split.setResizeWeight(0.78);
        split.setDividerSize(6);
        split.setBorder(null);
        root.add(split, BorderLayout.CENTER);

        // Widok parkingu
        JPanel widok = new JPanel(new BorderLayout(6, 6));
        widok.setBackground(Color.WHITE);

        // Sekcja samochodowa
        JPanel sek1 = new JPanel(new BorderLayout(4, 4));
        sek1.setBackground(CLR_TLO_SEKCJI);
        sek1.setBorder(BorderFactory.createCompoundBorder(

```

```

        new LineBorder(CLR_NAGLOWEK, 1), new EmptyBorder(6, 6, 6, 6)));
JLabel lab1 = naglowekSekcji(" Miejsca samochodowe");
sek1.add(lab1, BorderLayout.NORTH);
panelSamochodowe = new JPanel(new WrapLayout(FlowLayout.LEFT, 6, 6));
panelSamochodowe.setBackground(CLR_TLO_SEKCJI);
sek1.add(panelSamochodowe, BorderLayout.CENTER);

// Sekcja rowerowa
JPanel sek2 = new JPanel(new BorderLayout(4, 4));
sek2.setBackground(CLR_TLO_SEKCJI);
sek2.setBorder(BorderFactory.createCompoundBorder(
    new LineBorder(CLR_NAGLOWEK, 1), new EmptyBorder(6, 6, 6, 6)));
JLabel lab2 = naglowekSekcji(" Miejsca rowerowe");
sek2.add(lab2, BorderLayout.NORTH);
panelRowerowe = new JPanel(new WrapLayout(FlowLayout.LEFT, 6, 6));
panelRowerowe.setBackground(CLR_TLO_SEKCJI);
sek2.add(panelRowerowe, BorderLayout.CENTER);

JPanel sekcje = new JPanel(new GridLayout(2, 1, 0, 8));
sekcje.setBackground(Color.WHITE);
sekcje.add(sek1);
sekcje.add(sek2);

widok.add(sekcje, BorderLayout.CENTER);

// Legenda
widok.add(buildLegenda(), BorderLayout.SOUTH);
split.setLeftComponent(new JScrollPane(widok));

// Panel boczny: przyciski + raport
JPanel boczny = new JPanel(new BorderLayout(0, 10));
boczny.setBackground(Color.WHITE);

JPanel przyciski = buildPanelPrzyciskow();
boczny.add(przyciski, BorderLayout.NORTH);

JPanel panelRaport = new JPanel(new BorderLayout());
panelRaport.setBorder(BorderFactory.createCompoundBorder(
    BorderFactory.createTitledBorder(BorderFactory.createLineBorder(CLR_NAGLOWEK, 1), "
Raport "),
    new EmptyBorder(6, 6, 6, 6)));
panelRaport.setBackground(Color.WHITE);
labelRaport = new JLabel();
labelRaport.setVerticalAlignment(JLabel.TOP);
labelRaport.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 11));
panelRaport.add(labelRaport, BorderLayout.CENTER);
boczny.add(panelRaport, BorderLayout.CENTER);

split.setRightComponent(boczny);
}

private JLabel naglowekSekcji(String tekst) {
    JLabel l = new JLabel(tekst);
    l.setFont(new Font("SansSerif", Font.BOLD, 13));
    l.setForeground(CLR_NAGLOWEK);
    l.setBorder(new EmptyBorder(0, 0, 4, 0));
    return l;
}

private JPanel buildPanelPrzyciskow() {
    JPanel p = new JPanel(new GridLayout(0, 1, 0, 6));
    p.setBackground(Color.WHITE);
    p.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createTitledBorder(BorderFactory.createLineBorder(CLR_NAGLOWEK, 1), "
Operacje "),
        new EmptyBorder(6, 6, 6, 6)));

```

```

        JButton btnWjSam = new JButton("Wjazd samochodu");
        JButton btnWjRow = new JButton("Wjazd roweru");
        JButton btnWyj = new JButton("Wyjazd pojazdu");
        JButton btnOdswiez = new JButton("Odśwież widok");

        for (JButton b : new JButton[]{btnWjSam, btnWjRow, btnWyj, btnOdswiez}) {
            b.setFont(new Font("SansSerif", Font.PLAIN, 13));
            b.setFocusPainted(false);
            p.add(b);
        }

        btnWjSam.addActionListener(e -> { new DialogWjazdSamochodu(this,
parking).setVisible(true); odswiezWidok(); });
        btnWjRow.addActionListener(e -> { new DialogWjazdRoweru(this, parking).setVisible(true);
odswiezWidok(); });
        btnWyj.addActionListener(e -> { new DialogWyjazd(this, parking).setVisible(true);
odswiezWidok(); });
        btnOdswiez.addActionListener(e -> odswiezWidok());

        return p;
    }

    private JPanel buildLegenda() {
        JPanel p = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 4));
        p.setBackground(Color.WHITE);
        p.setBorder(new EmptyBorder(4, 0, 0, 0));
        p.add(legendaKlocek(CLR_WOLNE, "Wolne"));
        p.add(legendaKlocek(CLR_ZAJETE, "Zajęte"));
        p.add(legendaKlocek(CLR_ZABLOK, "Zablokowane"));
        p.add(legendaKlocek(CLR_LADOW, "Ładowarka EV"));
        p.add(legendaKlocek(CLR_ROWER_BL, "Blokada roweru"));
        return p;
    }

    private JPanel legendaKlocek(Color kolor, String opis) {
        JPanel p = new JPanel(new FlowLayout(FlowLayout.LEFT, 4, 0));
        p.setBackground(Color.WHITE);
        JPanel kwadrat = new JPanel();
        kwadrat.setPreferredSize(new Dimension(14, 14));
        kwadrat.setBackground(kolor);
        kwadrat.setBorder(new LineBorder(Color.GRAY, 1));
        p.add(kwadrat);
        JLabel l = new JLabel(opis);
        l.setFont(new Font("SansSerif", Font.PLAIN, 11));
        p.add(l);
        return p;
    }

    public void odswiezWidok() {
        odswiezSekcje();
        odswiezRaport();
    }

    private void odswiezSekcje() {
        panelSamochodowe.removeAll();
        for (MiejsceParkingowe m : parking.GetMiejscaSamochodowe()) {
            panelSamochodowe.add(buildKafelekSamochod(m));
        }
        panelSamochodowe.revalidate();
        panelSamochodowe.repaint();

        panelRowerowe.removeAll();
        for (MiejsceParkingowe m : parking.GetMiejscaRowerowe()) {
            panelRowerowe.add(buildKafelekRower(m));
        }
        panelRowerowe.revalidate();
    }

```

```

        panelRowerowe.repaint();
    }

    private JPanel buildKafelekSamochod(MiejsceParkingowe m) {
        JPanel k = new JPanel();
        k.setLayout(new BoxLayout(k, BoxLayout.Y_AXIS));
        k.setPreferredSize(new Dimension(118, 90));
        k.setBorder(new LineBorder(Color.DARK_GRAY, 1));

        Color tlo;
        if (!m.CzyCzynne()) tlo = CLR_ZABLOK;
        else if (m.CzyZajete()) tlo = CLR_ZAJETE;
        else if (m.CzyPosiadaLadowarke()) tlo = CLR_LADOW;
        else tlo = CLR_WOLNE;
        k.setBackground(tlo);

        String typ = m.CzyPosiadaLadowarke() ? "⚡ EV" : "Silnik";
        JLabel lNr = boldLabel("Nr " + m.getNrMiejsca() + " " + typ, 12);
        JLabel lStatus;
        JLabel lInfo1 = new JLabel(" ");
        JLabel lInfo2 = new JLabel(" ");
        lInfo1.setFont(new Font("SansSerif", Font.PLAIN, 10));
        lInfo2.setFont(new Font("SansSerif", Font.PLAIN, 10));

        if (m.CzyZajete() && m.getAktywnaSesja() != null) {
            SesjaParkingowa s = m.getAktywnaSesja();
            lStatus = new JLabel("ZAJĘTE");
            lStatus.setForeground(new Color(140, 0, 0));
            lStatus.setFont(new Font("SansSerif", Font.BOLD, 11));
            if (s.pojazd instanceof Samochod samochod) {
                lInfo1 = new JLabel(samochod.GetTablica());
                lInfo1.setFont(new Font("SansSerif", Font.PLAIN, 10));
            }
            if (s.getZaplanowanyCzasWyjazdu() != null) {
                lInfo2 = new JLabel("Do: " + FMT.format(s.getZaplanowanyCzasWyjazdu()));
                lInfo2.setFont(new Font("SansSerif", Font.PLAIN, 10));
            }
        } else if (!m.CzyCzynne()) {
            lStatus = new JLabel("ZABLOKOWANE");
            lStatus.setForeground(Color.DARK_GRAY);
            lStatus.setFont(new Font("SansSerif", Font.BOLD, 11));
        } else {
            lStatus = new JLabel("wolne");
            lStatus.setForeground(new Color(0, 100, 0));
            lStatus.setFont(new Font("SansSerif", Font.BOLD, 11));
        }

        for (JLabel l : new JLabel[]{lNr, lStatus, lInfo1, lInfo2}) {
            l.setAlignmentX(CENTER_ALIGNMENT);
            k.add(l);
        }
        k.add(Box.createVerticalGlue());
        return k;
    }

    private JPanel buildKafelekRower(MiejsceParkingowe m) {
        JPanel k = new JPanel();
        k.setLayout(new BoxLayout(k, BoxLayout.Y_AXIS));
        k.setPreferredSize(new Dimension(100, 70));
        k.setBorder(new LineBorder(Color.DARK_GRAY, 1));

        Color tlo;
        if (!m.CzyCzynne()) tlo = CLR_ZABLOK;
        else if (m.CzyZajete()) tlo = CLR_ZAJETE;
        else if (m.CzyPosiadaBlokade()) tlo = CLR_ROWER_BL;
        else tlo = CLR_WOLNE;
        k.setBackground(tlo);
    }

```

```

String typ = m.CzyPosiadaBlokade() ? "Rower z blokadą" : "Rower";
JLabel lNr = boldLabel("Nr " + m.getNrMiejsca() + " " + typ, 11);

JLabel lStatus;
JLabel lkWit = new JLabel(" ");
lKwit.setFont(new Font("SansSerif", Font.PLAIN, 10));

if (m.CzyZajete() && m.getAktywnaSesja() != null) {
    lStatus = new JLabel("ZAJĘTE");
    lStatus.setForeground(new Color(140, 0, 0));
    lStatus.setFont(new Font("SansSerif", Font.BOLD, 11));
    if (m.getAktywnaSesja().getNumerKwitu() > 0) {
        lkWit = new JLabel("Kwit: " + m.getAktywnaSesja().getNumerKwitu());
        lkWit.setFont(new Font("SansSerif", Font.PLAIN, 10));
    }
} else if (!m.CzyCzynne()) {
    lStatus = new JLabel("ZABLOK.");
    lStatus.setForeground(Color.DARK_GRAY);
    lStatus.setFont(new Font("SansSerif", Font.BOLD, 11));
} else {
    lStatus = new JLabel("wolne");
    lStatus.setForeground(new Color(0, 100, 0));
    lStatus.setFont(new Font("SansSerif", Font.BOLD, 11));
}

for (JLabel l : new JLabel[]{lNr, lStatus, lkWit}) {
    l.setAlignmentX(CENTER_ALIGNMENT);
    k.add(l);
}
k.add(Box.createVerticalGlue());
return k;
}

private JLabel boldLabel(String text, int size) {
    JLabel l = new JLabel(text);
    l.setFont(new Font("SansSerif", Font.BOLD, size));
    return l;
}

private void odswiezRaport() {
    Raporty r = parking.raporty;
    String html = "<html>" +
        "Miejsca łącznie: " + r.IloscMiejsc() + "\n" +
        "Wolne łącznie: " + r.IloscWolnychMiejsc() + "\n" +
        " - samochodowe: " + r.IloscWolnychMiejscSamoch() + "\n" +
        " - z ładowarką: " + r.IloscWolnychMiejscLadowarka() + "\n" +
        " - rowerowe: " + r.IloscWolnychMiejscRower() + "\n" +
        " - z blokadą: " + r.IloscWolnychMiejscBlokada() + "\n" +
        "Zarezerwowane: " + r.IloscMiejscZarezerwowanych() + "\n" +
        "Zablokowane: " + r.IloscMiejscZablokowanych() + "\n" +
        "_____\n" +
        "Zaparkowane łącznie: " + (r.IloscMiejsc() - r.IloscWolnychMiejsc()) + "\n" +
        " - samochody: " + r.IloscZaparkowanychSamoch() + "\n" +
        " - rowery: " + r.IloscZaparkowanychRower() +
        "</pre></html>";
    labelRaport.setText(html);
}

/**
 * FlowLayout z zawijaniem – miejsca parkingowe układają się w wiele rzędów
 * przy zmianie szerokości okna.
 */
static class WrapLayout extends FlowLayout {
    WrapLayout(int align, int hgap, int vgap) { super(align, hgap, vgap); }

    @Override

```

```

    public Dimension preferredLayoutSize(Container target) {
        return layoutSize(target, true);
    }

    @Override
    public Dimension minimumLayoutSize(Container target) {
        return layoutSize(target, false);
    }

    private Dimension layoutSize(Container target, boolean preferred) {
        synchronized (target.getTreeLock()) {
            int targetWidth = target.getSize().width;
            if (targetWidth == 0) targetWidth = Integer.MAX_VALUE;
            int hgap = getHgap(), vgap = getVgap();
            Insets insets = target.getInsets();
            int maxWidth = targetWidth - (insets.left + insets.right + hgap * 2);
            int width = 0, height = 0, rowWidth = 0, rowHeight = 0;
            int count = target.getComponentCount();
            for (int i = 0; i < count; i++) {
                Component c = target.getComponent(i);
                if (c.isVisible()) {
                    Dimension d = preferred ? c.getPreferredSize() : c.getMinimumSize();
                    if (rowWidth + d.width > maxWidth && rowWidth > 0) {
                        width = Math.max(width, rowWidth);
                        height += rowHeight + vgap;
                        rowWidth = 0; rowHeight = 0;
                    }
                    rowWidth += d.width + hgap;
                    rowHeight = Math.max(rowHeight, d.height);
                }
            }
            width = Math.max(width, rowWidth);
            height += rowHeight;
            return new Dimension(width + insets.left + insets.right + hgap * 2,
                height + insets.top + insets.bottom + vgap * 2);
        }
    }
}

```

DialogWjazdSamochodu.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import java.awt.*;
import java.time.Instant;
import java.time.temporal.ChronoUnit;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWjazdSamochodu extends JDialog {
    private final Parking parking;
    private JTextField fieldTablica;
    private JCheckBox checkElektryczny;
    private JSpinner spinnerGodziny;
    private JLabel labelWynik;

    public DialogWjazdSamochodu(Frame owner, Parking parking) {
        super(owner, "Wjazd pojazdu", true);
        this.parking = parking;
        buildUI();
        pack();
    }
}

```

```

        setLocationRelativeTo(owner);
    }

    private void buildUI() {
        setLayout(new BorderLayout(10, 10));
        getRootPane().setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));

        JPanel form = new JPanel(new GridBagLayout());
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(4, 4, 4, 4);
        c.anchor = GridBagConstraints.WEST;

        c.gridx = 0; c.gridy = 0;
        form.add(new JLabel("Nr rejestracyjny:"), c);
        c.gridx = 1; fieldTablica = new JTextField(12);
        form.add(fieldTablica, c);

        c.gridx = 0; c.gridy = 1;
        form.add(new JLabel("Pojazd elektryczny:"), c);
        c.gridx = 1; checkElektryczny = new JCheckBox();
        form.add(checkElektryczny, c);

        c.gridx = 0; c.gridy = 2;
        form.add(new JLabel("Planowany postój (godz.):"), c);
        c.gridx = 1;
        spinnerGodziny = new JSpinner(new SpinnerNumberModel(1, 1, 24, 1));
        form.add(spinnerGodziny, c);

        add(form, BorderLayout.CENTER);

        labelWynik = new JLabel(" ");
        labelWynik.setForeground(Color.RED);
        add(labelWynik, BorderLayout.NORTH);

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        JButton btnOK = new JButton("Zarejestruj wjazd");
        JButton btnAnuluj = new JButton("Anuluj");
        buttons.add(btnAnuluj);
        buttons.add(btnOK);
        add(buttons, BorderLayout.SOUTH);

        btnOK.addActionListener(e -> zarejestrujWjazd());
        btnAnuluj.addActionListener(e -> dispose());
    }

    private void zarejestrujWjazd() {
        String tablica = fieldTablica.getText().trim().toUpperCase();
        if (tablica.isEmpty()) {
            labelWynik.setText("Podaj numer rejestracyjny!");
            return;
        }

        int sprawdzenie = parking.sprawdzTabliceWjazd(tablica);
        if (sprawdzenie == -1) {
            parking.logi.ProbaWjazduZablokowanegoPojazdu(tablica);
            JOptionPane.showMessageDialog(this,
                "ODMOWA WJAZDU!\nTablica " + tablica + " jest na czarnej liście.",
                "Zablokowany pojazd", JOptionPane.ERROR_MESSAGE);
            return;
        }

        boolean elektryczny = checkElektryczny.isSelected();
        Samochod samochod = new Samochod(tablica, elektryczny);
        MiejsceParkingowe miejsce = parking.ZnajdzMiejsce(samochod);
    }

```



```

        if (miejsce == null) {
            labelWynik.setText("Brak wolnych miejsc samochodowych!");
            return;
        }

        int godziny = (int) spinnerGodziny.getValue();
        Instant wyjazd = Instant.now().plus(godziny, ChronoUnit.HOURS);
        parking.ZarejestrujWjazd(samochod, miejsce, wyjazd);

        String info = sprawdzenie == 1 ? " (klient zarejestrowany)" : " (klient
niezarejestrowany)";
        JOptionPane.showMessageDialog(this,
            "Wjazd zarejestrowany!\nNr rejestracyjny: " + tablica + info +
            "\nPrzydzielone miejsce nr: " + miejsce.getNrMiejsca() +
            (miejsce.CzyPosiadaLadowarke() ? " [ładowarka]" : ""),
            "Wjazd", JOptionPane.INFORMATION_MESSAGE);
        dispose();
    }
}

```

DialogWjazdRoweru.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import java.awt.*;
import java.time.Instant;
import java.time.temporal.ChronoUnit;
import java.util.Random;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWjazdRoweru extends JDialog {
    private final Parking parking;
    private JCheckBox checkWlasnaBlokada;
    private JLabel labelWynik;

    public DialogWjazdRoweru(Frame owner, Parking parking) {
        super(owner, "Wjazd roweru", true);
        this.parking = parking;
        buildUI();
        pack();
        setLocationRelativeTo(owner);
    }

    private void buildUI() {
        setLayout(new BorderLayout(10, 10));
        getRootPane().setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));

        JPanel form = new JPanel(new GridBagLayout());
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(4, 4, 4, 4);
        c.anchor = GridBagConstraints.WEST;

        c.gridx = 0; c.gridy = 0;
        form.add(new JLabel("Rowerzysta posiada własną blokadę:"), c);
        c.gridx = 1; checkWlasnaBlokada = new JCheckBox();
        form.add(checkWlasnaBlokada, c);

        add(form, BorderLayout.CENTER);
    }
}

```

```

        labelWynik = new JLabel(" ");
        labelWynik.setForeground(Color.RED);
        add(labelWynik, BorderLayout.NORTH);

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        JButton btnOK = new JButton("Zarejestruj wjazd");
        JButton btnAnuluj = new JButton("Anuluj");
        buttons.add(btnAnuluj);
        buttons.add(btnOK);
        add(buttons, BorderLayout.SOUTH);

        btnOK.addActionListener(e -> zarejestrujWjazd());
        btnAnuluj.addActionListener(e -> dispose());
    }

    private void zarejestrujWjazd() {
        boolean własnaBlokada = checkWlasnaBlokada.isSelected();
        Rower rower = new Rower();
        MiejsceParkingowe miejsce = parking.ZnajdzMiejsce(rower, własnaBlokada);

        if (miejsce == null) {
            labelWynik.setText("Brak wolnych miejsc rowerowych!");
            return;
        }

        int nrKwitu = 1000 + new Random().nextInt(9000);
        SesjaParkingowa sesja = new SesjaParkingowa(rower, Instant.now(),
            Instant.now().plus(8, ChronoUnit.HOURS), nrKwitu);
        miejsce.ZajmijMiejsce(sesja);
        parking.logi.WjazdRoweru(miejsce.getNrMiejsca());

        JOptionPane.showMessageDialog(this,
            "
            Kwit Parkingowy - Rower
            Nr miejsca: " + String.format("%-17d", miejsce.getNrMiejsca()) + "
            Nr kwitu: " + String.format("%-17d", nrKwitu) + "
            (miejsce.CzyPosiadaBlokade() ? " [blokada parkingowa] : " [własna
            blokada rowerzysty]
            "Wyдай rowerzyście ten kwit!",
            "Kwit parkingowy", JOptionPane.INFORMATION_MESSAGE);
        dispose();
    }
}

```

DialogWyjazd.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import java.awt.*;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWyjazd extends JDialog {
    private final Parking parking;
    private JTextField fieldTablicaLubKwit;
    private JRadioButton rbSamochod, rbRower;
    private JLabel labelWynik;

    public DialogWyjazd(Frame owner, Parking parking) {

```

```

        super(owner, "Wyjazd pojazdu", true);
        this.parking = parking;
        buildUI();
        pack();
        setLocationRelativeTo(owner);
    }

    private void buildUI() {
        setLayout(new BorderLayout(10, 10));
        getRootPane().setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));

        JPanel form = new JPanel(new GridBagLayout());
        GridBagConstraints c = new GridBagConstraints();
        c.insets = new Insets(4, 4, 4, 4);
        c.anchor = GridBagConstraints.WEST;

        c.gridx = 0; c.gridy = 0; c.gridwidth = 2;
        JPanel radioPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 6, 0));
        rbSamochod = new JRadioButton("Samochód", true);
        rbRower = new JRadioButton("Rower");
        ButtonGroup bg = new ButtonGroup();
        bg.add(rbSamochod); bg.add(rbRower);
        radioPanel.add(rbSamochod); radioPanel.add(rbRower);
        form.add(radioPanel, c);

        c.gridwidth = 1; c.gridy = 1; c.gridx = 0;
        JLabel labelPole = new JLabel("Nr rejestracyjny:");
        form.add(labelPole, c);
        c.gridx = 1;
        fieldTablicaLubKwit = new JTextField(14);
        form.add(fieldTablicaLubKwit, c);

        rbSamochod.addActionListener(e -> labelPole.setText("Nr rejestracyjny:"));
        rbRower.addActionListener(e -> labelPole.setText("Nr kwitu:"));

        add(form, BorderLayout.CENTER);

        labelWynik = new JLabel(" ");
        labelWynik.setForeground(Color.RED);
        add(labelWynik, BorderLayout.NORTH);

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        JButton btnOK = new JButton("Zarejestruj wyjazd");
        JButton btnAnuluj = new JButton("Anuluj");
        buttons.add(btnAnuluj); buttons.add(btnOK);
        add(buttons, BorderLayout.SOUTH);

        btnOK.addActionListener(e -> zarejestrujWyjazd());
        btnAnuluj.addActionListener(e -> dispose());
    }

    private void zarejestrujWyjazd() {
        String wartoscPola = fieldTablicaLubKwit.getText().trim();
        if (wartoscPola.isEmpty()) {
            labelWynik.setText("Uzupełnij pole!");
            return;
        }

        if (rbSamochod.isSelected()) {
            String tablica = wartoscPola.toUpperCase();
            MiejsceParkingowe miejsce = parking.ZnajdzMiejscePojazdaZTabl(tablica);
            if (miejsce == null) {
                labelWynik.setText("Nie znaleziono samochodu o tablicy " + tablica);
                return;
            }
            int nrMiejsca = miejsce.getNrMiejsca();

```

```

        parking.ZarejestrujWyjazd(miejsce);
        JOptionPane.showMessageDialog(this,
            "Wyjazd zarejestrowany.\nSamochód " + tablica + " opuścił miejsce nr " +
nrMiejsca,
            "Wyjazd", JOptionPane.INFORMATION_MESSAGE);
        dispose();
    } else {
        // Rower - szukaj po numerze kwitu
        int nrKwitu;
        try { nrKwitu = Integer.parseInt(wartoscPola); }
        catch (NumberFormatException ex) { labelWynik.setText("Nr kwitu musi być liczbą!");
return; }

        MiejsceParkingowe znalezione = null;
        for (MiejsceParkingowe m : parking.GetMiejscaRowerowe()) {
            if (m.CzyZajete() && m.getAktywnaSesja() != null
                && m.getAktywnaSesja().getNumerKwitu() == nrKwitu) {
                znalezione = m;
                break;
            }
        }
        if (znalezione == null) {
            labelWynik.setText("Nie znaleziono roweru z kwitem nr " + nrKwitu);
            return;
        }
        int nrMiejsca = znalezione.getNrMiejsca();
        parking.ZarejestrujWyjazd(znalezione);
        JOptionPane.showMessageDialog(this,
            "Wyjazd roweru zarejestrowany.\nMiejsce nr " + nrMiejsca + " zwolnione.",
            "Wyjazd roweru", JOptionPane.INFORMATION_MESSAGE);
        dispose();
    }
}
}
}

```

DialogDodajKlienta.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import java.awt.*;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogDodajKlienta extends JDialog {
    private final Parking parking;
    private JTextField fieldTablica, fieldImie, fieldEmail, fieldTelefon;
    private JLabel labelWynik;

    public DialogDodajKlienta(Frame owner, Parking parking) {
        super(owner, "Dodaj klienta (rejestracja tablicy)", true);
        this.parking = parking;
        buildUI();
        pack();
        setMinimumSize(new Dimension(380, 0));
        setLocationRelativeTo(owner);
    }

    private void buildUI() {
        setLayout(new BorderLayout(10, 10));
        getRootPane().setBorder(BorderFactory.createEmptyBorder(12, 12, 12, 12));

        JPanel form = new JPanel(new GridBagLayout());
    }

```

```

GridBagConstraints c = new GridBagConstraints();
c.insets = new Insets(4, 4, 4, 4);
c.anchor = GridBagConstraints.WEST;
c.fill = GridBagConstraints.HORIZONTAL;

String[] labels = {"Nr rejestracyjny:", "Imię i nazwisko:", "Adres e-mail:", "Nr
telefonu:"};
JTextField[] fields = new JTextField[4];
for (int i = 0; i < 4; i++) {
    c.gridx = 0; c.gridy = i; c.weightx = 0;
    form.add(new JLabel(labels[i]), c);
    c.gridx = 1; c.weightx = 1;
    fields[i] = new JTextField(16);
    form.add(fields[i], c);
}
fieldTablica = fields[0];
fieldImie = fields[1];
fieldEmail = fields[2];
fieldTelefon = fields[3];

add(form, BorderLayout.CENTER);

labelWynik = new JLabel(" ");
labelWynik.setForeground(Color.RED);
add(labelWynik, BorderLayout.NORTH);

JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
JButton btnOK = new JButton("Dodaj");
JButton btnAnuluj = new JButton("Anuluj");
buttons.add(btnAnuluj); buttons.add(btnOK);
add(buttons, BorderLayout.SOUTH);

btnOK.addActionListener(e -> dodaj());
btnAnuluj.addActionListener(e -> dispose());
}

private void dodaj() {
    String tablica = fieldTablica.getText().trim().toUpperCase();
    String imie = fieldImie.getText().trim();
    String email = fieldEmail.getText().trim();
    String tel = fieldTelefon.getText().trim();

    if (tablica.isEmpty() || imie.isEmpty()) {
        labelWynik.setText("Tablica i imię/nazwisko są wymagane!");
        return;
    }
    if (tel.length() != 9 || !tel.matches("\\d+")){
        labelWynik.setText("Niepoprawny numer telefonu");
        return;
    }
    if (!email.contains("@")){
        labelWynik.setText("Niepoprawny email!");
        return;
    }
    if (parking.zarejestrowaneTablice.CzyZablokowana(tablica)) {
        JOptionPane.showMessageDialog(this,
            "Nie można dodać klienta – tablica " + tablica + " jest na czarnej liście!",
            "Tablica zablokowana", JOptionPane.ERROR_MESSAGE);
        return;
    }

    Wlasciciel w = new Wlasciciel(imie, email, tel);
    parking.DodajKlienta(tablica, w);

    JOptionPane.showMessageDialog(this,

```

```

        "Klient zarejestrowany!\nTablica: " + tablica + "\n" + w.Dane(),
        "Sukces", JOptionPane.INFORMATION_MESSAGE);
    dispose();
}
}

```

DialogWyszukajKlienta.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import javax.swing.table.*;
import java.awt.*;
import java.util.Map;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogWyszukajKlienta extends JDialog {
    private final Parking parking;
    private JTextField fieldSzukaj;
    private DefaultTableModel tableModel;
    private JTable tabela;

    public DialogWyszukajKlienta(Frame owner, Parking parking) {
        super(owner, "Wyszukiwanie klientów", true);
        this.parking = parking;
        buildUI();
        odswiezTabele("");
        setSize(600, 400);
        setLocationRelativeTo(owner);
    }

    private void buildUI() {
        setLayout(new BorderLayout(8, 8));
        getRootPane().setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        JPanel top = new JPanel(new BorderLayout(6, 0));
        top.add(new JLabel("Szukaj (tablica/imię/email/tel): "), BorderLayout.WEST);
        fieldSzukaj = new JTextField();
        top.add(fieldSzukaj, BorderLayout.CENTER);
        JButton btnSzukaj = new JButton("Szukaj");
        top.add(btnSzukaj, BorderLayout.EAST);
        add(top, BorderLayout.NORTH);

        String[] cols = {"Tablica", "Imię i nazwisko", "E-mail", "Telefon"};
        tableModel = new DefaultTableModel(cols, 0) {
            public boolean isCellEditable(int r, int c) { return false; }
        };
        tabela = new JTable(tableModel);
        tabela.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        add(new JScrollPane(tabela), BorderLayout.CENTER);

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        JButton btnEdytuj = new JButton("Edytuj");
        JButton btnUsun = new JButton("Usuń");
        JButton btnZamknij = new JButton("Zamknij");
        buttons.add(btnEdytuj); buttons.add(btnUsun); buttons.add(btnZamknij);
        add(buttons, BorderLayout.SOUTH);

        fieldSzukaj.addActionListener(e -> odswiezTabele(fieldSzukaj.getText()));
        btnSzukaj.addActionListener(e -> odswiezTabele(fieldSzukaj.getText()));
        btnEdytuj.addActionListener(e -> edytujWybrany());
    }

```

```

        btnUsun.addActionListener(e -> usunWybrany());
        btnZamknij.addActionListener(e -> dispose());
    }

    private void odswiezTabele(String fraza) {
        tableModel.setRowCount(0);
        String f = fraza.trim().toLowerCase();
        for (Map.Entry<String, Wlasciciel> e :
parking.zarejestrowaneTablice.GetWszystkieTablice().entrySet()) {
            String tab = e.getKey();
            Wlasciciel w = e.getValue();
            if (f.isEmpty() || tab.toLowerCase().contains(f)
                || w.GetImieNazwisko().toLowerCase().contains(f)
                || w.GeteMail().toLowerCase().contains(f)
                || w.GetNrTelefonu().contains(f)) {
                tableModel.addRow(new Object[]{tab, w.GetImieNazwisko(), w.GeteMail(),
w.GetNrTelefonu()});
            }
        }
    }

    private void edytujWybrany() {
        int row = tabela.getSelectedRow();
        if (row < 0) { JOptionPane.showMessageDialog(this, "Wybierz klienta z listy."); return; }
        String tablica = (String) tableModel.getValueAt(row, 0);
        Wlasciciel w = parking.ZnajdzWlasciciela(tablica);
        if (w == null) return;

        JTextField fImie = new JTextField(w.GetImieNazwisko(), 18);
        JTextField fEmail = new JTextField(w.GeteMail(), 18);
        JTextField fTel = new JTextField(w.GetNrTelefonu(), 18);
        JPanel p = new JPanel(new GridLayout(3, 2, 4, 4));
        p.add(new JLabel("Imię i nazwisko:")); p.add(fImie);
        p.add(new JLabel("E-mail:")); p.add(fEmail);
        p.add(new JLabel("Telefon:")); p.add(fTel);

        int res = JOptionPane.showConfirmDialog(this, p, "Edycja klienta " + tablica,
JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE);
        if (res == JOptionPane.OK_OPTION) {
            // aktualizuj() zamiast trzech setterów
            w.Aktualizuj(fImie.getText().trim(), fEmail.getText().trim(), fTel.getText().trim());
            odswiezTabele(fieldSzukaj.getText());
        }
    }

    private void usunWybrany() {
        int row = tabela.getSelectedRow();
        if (row < 0) { JOptionPane.showMessageDialog(this, "Wybierz klienta z listy."); return; }
        String tablica = (String) tableModel.getValueAt(row, 0);
        int res = JOptionPane.showConfirmDialog(this,
            "Czy na pewno usunąć klienta z tablicą " + tablica + "?",
            "Potwierdzenie", JOptionPane.YES_NO_OPTION);
        if (res == JOptionPane.YES_OPTION) {
            parking.UsunKlienta(tablica);
            odswiezTabele(fieldSzukaj.getText());
        }
    }
}

```

DialogCzarnaLista.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import javax.swing.table.*;

```

```

import java.awt.*;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogCzarnaLista extends JDialog {
    private final Parking parking;
    private DefaultTableModel tableModel;
    private JTable tabela;
    private JTextField fieldSzukaj;

    public DialogCzarnaLista(Frame owner, Parking parking) {
        super(owner, "Czarna lista tablic", true);
        this.parking = parking;
        buildUI();
        odswiezTabele("");
        setSize(450, 380);
        setLocationRelativeTo(owner);
    }

    private void buildUI() {
        setLayout(new BorderLayout(8, 8));
        getRootPane().setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        JPanel top = new JPanel(new BorderLayout(6, 4));
        JPanel addPanel = new JPanel(new BorderLayout(4, 0));
        JTextField fieldNowa = new JTextField(12);
        JButton btnDodaj = new JButton("Dodaj do czarnej listy");
        addPanel.add(new JLabel("Tablica: "), BorderLayout.WEST);
        addPanel.add(fieldNowa, BorderLayout.CENTER);
        addPanel.add(btnDodaj, BorderLayout.EAST);

        JPanel szukajPanel = new JPanel(new BorderLayout(4, 0));
        fieldSzukaj = new JTextField(12);
        JButton btnSzukaj = new JButton("Szukaj");
        szukajPanel.add(new JLabel("Szukaj: "), BorderLayout.WEST);
        szukajPanel.add(fieldSzukaj, BorderLayout.CENTER);
        szukajPanel.add(btnSzukaj, BorderLayout.EAST);

        top.add(addPanel, BorderLayout.NORTH);
        top.add(szukajPanel, BorderLayout.SOUTH);
        add(top, BorderLayout.NORTH);

        tableModel = new DefaultTableModel(new String[]{"Zablokowana tablica"}, 0) {
            public boolean isCellEditable(int r, int c) { return false; }
        };
        tabela = new JTable(tableModel);
        tabela.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
        add(new JScrollPane(tabela), BorderLayout.CENTER);

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        JButton btnUsun = new JButton("Usuń z czarnej listy");
        JButton btnZamknij = new JButton("Zamknij");
        buttons.add(btnUsun); buttons.add(btnZamknij);
        add(buttons, BorderLayout.SOUTH);

        btnDodaj.addActionListener(e -> {
            String tab = fieldNowa.getText().trim().toUpperCase();
            if (!tab.isEmpty()) {
                // ZablokujTablice automatycznie usuwa pojazd z parkingu jeśli jest zaparkowany
                parking.zarejestrowaneTablice.ZablokujTablice(tab);
                fieldNowa.setText("");
                odswiezTabele(fieldSzukaj.getText());
            }
        });
        fieldNowa.addActionListener(e -> btnDodaj.doClick());
    }
}

```



```

        btnSzukaj.addActionListener(e -> odswiezTabele(fieldSzukaj.getText()));
        fieldSzukaj.addActionListener(e -> odswiezTabele(fieldSzukaj.getText()));
        btnUsun.addActionListener(e -> usunWybrany());
        btnZamknij.addActionListener(e -> dispose());
    }

    private void odswiezTabele(String fraza) {
        tableModel.setRowCount(0);
        String f = fraza.trim().toLowerCase();
        for (String tab : parking.zarejestrowaneTablice.GetZablokowaneTablice()) {
            if (f.isEmpty() || tab.toLowerCase().contains(f))
                tableModel.addRow(new Object[]{tab});
        }
    }

    private void usunWybrany() {
        int row = tabela.getSelectedRow();
        if (row < 0) { JOptionPane.showMessageDialog(this, "Wybierz tablicę z listy."); return; }
        String tab = (String) tableModel.getValueAt(row, 0);
        int res = JOptionPane.showConfirmDialog(this,
            "Usunąć tablicę " + tab + " z czarnej listy?",
            "Potwierdzenie", JOptionPane.YES_NO_OPTION);
        if (res == JOptionPane.YES_OPTION) {
            parking.zarejestrowaneTablice.OdblokujTablice(tab);
            odswiezTabele(fieldSzukaj.getText());
        }
    }
}

```

DialogEksportLogow.java

```

package com.systemzarzadzaniaparkingiem.widok;

import com.systemzarzadzaniaparkingiem.model.*;
import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

/**
 *
 * @author Jakub Kanecki, Dawid Lazarski
 */
public class DialogEksportLogow extends JDialog {
    private final Parking parking;
    private JTextArea areaLogi;

    public DialogEksportLogow(Frame owner, Parking parking) {
        super(owner, "Logi systemu - eksport", true);
        this.parking = parking;
        buildUI();
        setSize(620, 480);
        setLocationRelativeTo(owner);
    }

    private void buildUI() {
        setLayout(new BorderLayout(8, 8));
        getRootPane().setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

        areaLogi = new JTextArea();
        areaLogi.setEditable(false);
        areaLogi.setFont(new Font(Font.MONOSPACED, Font.PLAIN, 12));
        areaLogi.setText(parking.logi.Serializuj());
        add(new JScrollPane(areaLogi), BorderLayout.CENTER);
    }
}

```

```

        JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        JButton btnOdswiez = new JButton("Odśwież");
        JButton btnEksport = new JButton("Eksportuj do TXT...");
        JButton btnZamknij = new JButton("Zamknij");
        buttons.add(btnOdswiez); buttons.add(btnEksport); buttons.add(btnZamknij);
        add(buttons, BorderLayout.SOUTH);

        btnOdswiez.addActionListener(e -> areaLogi.setText(parking.logi.Serializuj()));
        btnEksport.addActionListener(e -> eksportuj());
        btnZamknij.addActionListener(e -> dispose());
    }

    private void eksportuj() {
        String domyslna = "logi_parkingu_" +
            LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmss")) + ".txt";
        JFileChooser fc = new JFileChooser();
        fc.setSelectedFile(new File(domyslna));
        if (fc.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;

        File plik = fc.getSelectedFile();
        try (PrintWriter pw = new PrintWriter(new OutputStreamWriter(
            new FileOutputStream(plik), StandardCharsets.UTF_8))) {
            pw.print(parking.logi.Serializuj());
            parking.logi.EksportLogow(plik.getAbsolutePath());
            areaLogi.setText(parking.logi.Serializuj());
            JOptionPane.showMessageDialog(this,
                "Logi zapisane do:\n" + plik.getAbsolutePath(),
                "Eksport zakończony", JOptionPane.INFORMATION_MESSAGE);
        } catch (IOException ex) {
            JOptionPane.showMessageDialog(this,
                "Błąd zapisu pliku:\n" + ex.getMessage(),
                "Błąd", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

Pakiet główny com.systemzarzadzaniaparkingiem

SystemZarzadzaniaParkingiem.java

```

/**
 * @autorzy: Jakub Kanecki, Dawid Łazarski
 */
package com.systemzarzadzaniaparkingiem;

import com.systemzarzadzaniaparkingiem.model.Parking;
import com.systemzarzadzaniaparkingiem.widok.MainWindow;
import javax.swing.UIManager;

public class SystemZarzadzaniaParkingiem {

    public static void main(String[] args) {
        try {
            for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }

        // 2. start GUI w EDT
    }
}

```

```
java.awt.EventQueue.invokeLater(() -> {  
    MainWindow widok = new MainWindow();  
  
    widok.setVisible(true);  
});  
  
}
```

Przypominam , że dokumentacje zapisujecie Państwo w pliku o rozszerzeniu *.pdf.
Proszę o ujednolicone nazewnictwo wg. szablonu.

dokumentacja: **PO_Pr_dok_Grxx.pdf**

np. PO_Pr_dok_Gr2.pdf

katalog programu: **PO_Pr_kod_Grxx.zip**

np. PO_Pr_kod_Gr2.zip

Sposób przekazania dokumentów ustala prowadzący. (ePortal)